

2 Tổng quan về hệ thống song song

Tóm tắt chương

Trong chương này chúng tôi tổng quan về các khái niệm, khái niệm cơ bản quan trọng nhất và kết quả lý thuyết liên quan đến tính toán song song. Chúng tôi mô tả ba mô hình cơ bản tính toán song song, sau đó tập trung vào các cấu trúc liên kết khác nhau để kết nối các nút máy tính song song. Sau đây là phần giới thiệu ngắn gọn về phân tích quan hệ song song phức tạp của cách đặt, cuối cùng chúng tôi giải thích hai định luật quan trọng của tính toán song song, định luật Amdahl và định lý Brents.

2.1 Lịch sử tính toán, hệ thống và lập trình song song

Giả sử Π là một bài toán tính toán tùy ý được giải bằng máy tính.

Thông thường mục tiêu đầu tiên của chúng ta là thiết kế một thuật toán để giải Π . Rõ ràng, lớp của tất cả các thuật toán là vô hạn, nhưng chúng ta có thể phân chia nó thành hai lớp con, lớp tất cả các thuật toán tuần tự và lớp của tất cả các thuật toán song song.¹ Trong khi một thuật toán tuần tự thực hiện một thao tác trong mỗi bước, thuật toán song song có thể thực hiện nhiều thao tác trong một bước duy nhất. Trong cuốn sách này, chúng ta sẽ chủ yếu quan tâm đến các thuật toán song song. Vì vậy, mục tiêu của chúng tôi là thiết kế một thuật toán song song cho Π .

Cho P là một thuật toán song song tùy ý. Chúng ta nói rằng có sự song song trong P . Tính song song trong P có thể được khai thác bởi nhiều loại máy tính song song.

Ví dụ, nhiều thao tác của P có thể được thực hiện đồng thời bởi nhiều bộ xử lý của máy tính song song $C1$; hoặc, có lẽ, chúng có thể bị xử lý từ bởi nhiều đơn vị chức năng được liên kết với nhau của một máy tính có bộ xử lý đơn $C2$. Rốt cuộc thì P .

¹ Ngoài ra còn có các bộ phận khác phân chia lớp của tất cả các thuật toán theo các tiêu chí khác, chẳng hạn như các thuật toán chính xác và không chính xác; thuật toán xác định và không xác định. Tuy nhiên, trong cuốn sách này chúng tôi sẽ không phân chia các thuật toán một cách có hệ thống theo những tiêu chí này.

luôn có thể được thực hiện tuần tự trên máy tính có bộ xử lý đơn C3, chỉ bằng cách thực hiện lần lượt các hoạt động có khả năng song song của P. Giả sử C(p) là một máy tính song song loại C chứa p đơn vị xử lý. Đương nhiên, chúng ta kỳ vọng hiệu năng của P trên C(p) sẽ phụ thuộc cả vào C và p. Chúng tôi do đó, phải phân biệt rõ ràng giữa tiềm năng song song P với một bên và khả năng thực tế của C(p) để thực hiện song song nhiều thao tác của P, ở phía bên kia. Vậy hiệu năng của thuật toán P trên máy tính song song C(p) phụ thuộc vào khả năng của C(p) trong việc khai thác tính song song tiềm tàng của P. Trước khi tiếp tục, chúng ta phải xác định rõ ràng ý nghĩa thực sự của từ này. Thuật ngữ "hiệu suất" của thuật toán song song P. Theo trực giác, "hiệu suất" có thể có nghĩa là thời gian cần thiết để thực hiện P trên C(p); đây được gọi là thực thi song song thời gian (hoặc thời gian chạy song song) của P trên C(p), mà chúng ta sẽ biểu thị bằng

T_{par} .

Ngoài ra, chúng ta có thể chọn "hiệu suất" có nghĩa là số lần thực thi song song P trên C(p) nhanh hơn thực thi tuần tự P; đây là gọi là sự tăng tốc của P trên C(p),

$$S_{def} = \frac{T_{seq}}{T_{par}}$$

Vì vậy, việc thực thi song song P trên C(p) nhanh hơn S lần so với thực thi tuần tự của P. Tiếp theo, chúng ta có thể quan tâm đến mức độ tăng tốc trung bình của S là bao nhiêu, do từng đơn vị xử lý. Nói cách khác, thuật ngữ "hiệu suất" có thể được hiểu là mức đóng góp trung bình của mỗi đơn vị xử lý p của C(p) để tăng tốc; đây được gọi là hiệu quả của P trên C(p),

$$E_{def} = \frac{S}{p}$$

Vì $T_{par} \leq T_{seq} \leq p \cdot T_{par}$, nên việc tăng tốc bị giới hạn ở trên bởi p và hiệu suất ciency được giới hạn ở trên bởi

$E \leq 1$.

Điều này có nghĩa là, với mọi C và p, việc thực hiện song song P trên C(p) có thể nhiều nhất là nhanh hơn p lần so với việc thực thi P trên một bộ xử lý. Và hiệu quả của việc thực thi song song P trên C(p) có thể nhiều nhất là 1. (Đây là khi mỗi đơn vị xử lý liên tục tham gia vào việc thực hiện P, do đó đóng góp 1

p-th để tăng tốc độ của nó.)

Sau đó, trong Phần 2.5, chúng tôi sẽ đưa thêm một tham số nữa vào các định nghĩa này.

Từ các định nghĩa trên chúng ta thấy rằng cả tốc độ và hiệu quả đều phụ thuộc vào T_{par} , thời gian thực hiện song song của P trên C(p). Điều này đặt ra những câu hỏi mới:

Làm thế nào để chúng tôi xác định Tpar?
Tpar phụ thuộc vào C (loại máy tính song song) như thế nào?
Những tính chất nào của C chúng ta phải tính đến để xác định Tpar?

Đây là những câu hỏi chung quan trọng về tính toán song song cần phải được giải đáp. đã trả lời trước khi bắt tay vào thiết kế và phân tích thực tế các thuật toán song song. Cách trả lời những câu hỏi này là mô hình hóa tính toán song song một cách thích hợp.

2.2 Mô hình hóa tính toán song song

Các máy tính song song có sự khác nhau rất lớn trong cách tổ chức của chúng. Chúng ta sẽ thấy trong phần tiếp theo rằng các đơn vị xử lý của họ có thể được kết nối trực tiếp hoặc không với nhau; một số trong số các đơn vị xử lý có thể chia sẻ bộ nhớ chung trong khi các đơn vị khác chỉ có thể sở hữu ký ức địa phương (riêng tư); hoạt động của các đơn vị xử lý có thể được đồng bộ hóa bởi một chiếc đồng hồ chung, hoặc chúng có thể chạy theo tốc độ riêng của nó. Hơn nữa, thường có là các chi tiết kiến trúc và đặc tính phần cứng của các thành phần, tất cả đều thể hiện trong quá trình thiết kế và sử dụng máy tính thực tế. Và cuối cùng, có công nghệ sự khác biệt, biểu hiện ở tốc độ xung nhịp, thời gian truy cập bộ nhớ khác nhau, v.v. câu hỏi sau đây phát sinh:

Những thuộc tính nào của máy tính song song phải được xem xét và điều gì có thể bị bỏ qua trong quá trình thiết kế và phân tích các thuật toán song song?

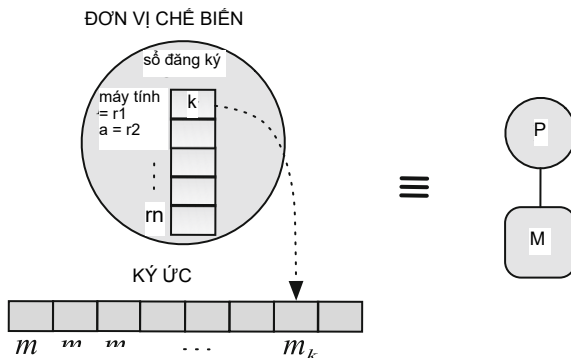
Để trả lời câu hỏi, chúng tôi áp dụng những ý tưởng tương tự như những ý tưởng được phát hiện trong trường hợp tính toán tuần tự. Ở đó, nhiều mô hình tính toán khác nhau đã được phát hiện.²

Nói tóm lại, mục đích của mỗi mô hình này là trừu tượng hóa các thuộc tính liên quan của phép tính (tuần tự) từ những phép tính không liên quan.

Trong trường hợp của chúng tôi, một mô hình có tên là Máy truy cập ngẫu nhiên (RAM) đặc biệt hấp dẫn. Tại sao? Lý do là RAM chất lọc những thuộc tính quan trọng của các máy tính tuần tự có mục đích chung, vẫn được sử dụng rộng rãi cho đến ngày nay, và thực sự đã được lấy làm cơ sở khái niệm cho việc mô hình hóa song song tính toán và máy tính song song. Hình 2.1 thể hiện cấu trúc của RAM.

²Một số mô hình tính toán này là hàm μ -đệ quy, hàm đệ quy, phép tính λ , Máy Turing, máy Post, thuật toán Markov và RAM.

Hình 2.1 Mô hình RAM của
tính toán có bộ nhớ
M (chứa chương trình
hướng dẫn và dữ liệu) và một
bộ xử lý P (thực thi
hướng dẫn về dữ liệu)



Dưới đây là mô tả ngắn gọn về RAM:

- RAM bao gồm bộ xử lý và bộ nhớ. Trị nhớ là một tiềm năng chuỗi **vô hạn các vị trí có kích thước bằng nhau** $m_0, m_1, \dots$. Chỉ số i được gọi địa **chỉ của mi**. Mỗi vị trí có thể **được truy cập trực tiếp** bởi đơn vị xử lý: được cung cấp một i tùy ý, việc đọc từ m_i hoặc viết vào m_i được thực hiện trong thời gian không đổi. Các thanh ghi là một dãy $r_1 \dots r_n$ của các vị trí trong đơn vị xử lý. Số đăng ký là có thể truy cập trực tiếp. Hai trong số họ có vai trò đặc biệt. Bộ đếm chương trình $pc (=r_1)$ chứa địa chỉ của vị trí trong bộ nhớ chứa lệnh sẽ được thực hiện tiếp theo. Bộ tích lũy $a (=r_2)$ tham gia vào việc thực thi từng hướng dẫn. Các thanh ghi khác được giao vai trò khi cần thiết. Chương trình là một chương trình tự các lệnh (tương tự như trong máy tính thực).
- Trước khi khởi động RAM, việc sau được thực hiện: (a) một chương trình được tải vào các vị trí liên tiếp của bộ nhớ bắt đầu bằng m_0 ; (b) dữ liệu đầu vào được ghi vào các vị trí bộ nhớ trống, chẳng hạn như sau lệnh chương trình cuối cùng.
- Từ nay trở đi, RAM hoạt động độc lập theo kiểu cơ học từng bước theo hướng dẫn của chương trình. Đặt $pc = k$ ở đầu bước. (Ban đầu, $k = 0$.) Từ vị trí m_k , lệnh l được đọc và bắt đầu. Đồng thời thời gian, pc được tăng lên. Vì vậy, khi tôi hoàn thành, hướng dẫn tiếp theo sẽ là được thực thi theo m_{k+1} , trừ khi tôi là một trong những hướng dẫn thay đổi pc (ví dụ: hướng dẫn nhảy).

Vì vậy, câu hỏi trên được rút gọn thành câu hỏi sau:

Mô hình tính toán song song thích hợp là gì?

Hóa ra việc tìm ra câu trả lời cho câu hỏi này khó khăn hơn rất nhiều hơn so với trường hợp tính toán tuần tự. Tại sao? Vì có nhiều cách để tổ chức các máy tính song song cũng có nhiều cách để mô hình hóa chúng; và cái gì là khó khăn là chọn một mô hình duy nhất phù hợp cho tất cả các máy tính song song.

Kết quả là, trong những thập kỷ qua, các nhà nghiên cứu đã đề xuất một số mô hình song song tính toán. Tuy nhiên, chưa có thỏa thuận chung nào được đưa ra về việc cài đung. Trong phần sau đây, chúng tôi mô tả những phần mềm dựa trên RAM.³

2.3 Mô hình đa bộ xử lý

Mô hình đa bộ xử lý là mô hình tính toán song song được xây dựng trên RAM mô hình tính toán; nghĩa là nó khái quát hóa RAM. Làm thế nào nó làm được điều đó? Hóa ra việc khái quát hóa có thể được thực hiện theo ba cách cơ bản khác nhau dẫn đến ba mô hình đa bộ xử lý khác nhau. Mỗi trong số ba mô hình có một số số p ($\square 2$) của các đơn vị xử lý, nhưng các mô hình khác nhau về cách tổ chức ký ức và cách các đơn vị xử lý truy cập vào ký ức. Các mô hình được gọi là

- Máy truy cập ngẫu nhiên song song (PRAM),
- Máy bộ nhớ cục bộ (LMM) và
- Máy bộ nhớ mô-đun (MMM).

Hãy để chúng tôi mô tả chúng.

2.3.1 Máy truy cập ngẫu nhiên song song

Máy truy cập ngẫu nhiên song song, viết tắt là mô hình PRAM, có khả năng xử lý p tất cả các đơn vị đều được kết nối với một bộ nhớ dùng chung không giới hạn (Hình 2.2).

Mỗi đơn vị xử lý có thể, trong một bước, truy cập bất kỳ vị trí (word) nào trong bộ nhớ dùng chung bằng cách đưa ra yêu cầu bộ nhớ trực tiếp tới bộ nhớ dùng chung.

Mô hình tính toán song song PRAM được lý tưởng hóa ở một số khía cạnh. Đầu tiên, không có giới hạn về số lượng đơn vị xử lý p , ngoại trừ p là hữu hạn. Tiếp theo, cũng lý tưởng hơn là giả định rằng một đơn vị xử lý có thể truy cập bất kỳ vị trí nào trong bộ nhớ chia sẻ trong một bước duy nhất. Cuối cùng, đối với các từ trong bộ nhớ dùng chung thì chỉ

giả định rằng chúng có cùng kích thước; nếu không thì chúng có thể có kích thước hữu hạn tùy ý.

Lưu ý rằng trong mô hình này không có mạng kết nối để truyền bộ nhớ các yêu cầu và dữ liệu qua lại giữa các đơn vị xử lý và bộ nhớ dùng chung.

(Điều này sẽ thay đổi hoàn toàn trong hai mô hình còn lại, LMM (xem Phần 2.3.2) và MMM (xem Phần 2.3.3)).

Tuy nhiên, giả định rằng bất kỳ đơn vị xử lý nào cũng có thể truy cập vào bất kỳ vị trí bộ nhớ nào trong một bước là không thực tế. Để biết lý do tại sao, giả sử rằng các đơn vị xử lý P_i và P_j

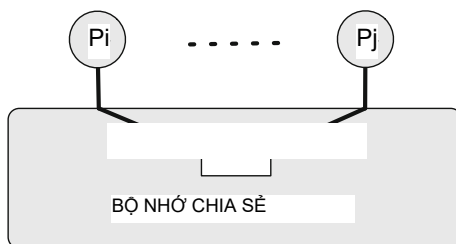
³Trên thực tế, hiện nay nghiên cứu cũng đang được theo đuổi theo các hướng khác, không theo thông lệ, mà không xây dựng trên RAM hoặc bất kỳ mô hình tính toán thông thường nào khác (được liệt kê trong chú thích cuối trang trước).

Ví dụ như tính toán luồng dữ liệu và tính toán lượng tử.

Hình.2.2 Mô hình PRAM
tính toán song song: p
các đơn vị xử lý chia sẻ một
bộ nhớ không giới hạn. Mỗi
đơn vị xử lý có thể trong một
bước truy cập bất kỳ bộ nhớ
vị trí
BỘ NHỚ CHIA SẺ



Hình.2.3 Mối nguy hiểm của
truy cập đồng thời vào một
vị trí. Hai chế biến
đơn vị đồng loạt phát hành
từng hướng dẫn đó
cần truy cập tương tự
vị trí L



đồng thời đưa ra các hướng dẫn li và lj nơi cả hai hướng dẫn đều có ý định truy cập (để đọc hoặc ghi vào) cùng một vị trí bộ nhớ L (xem Hình.2.3). Ngay cả khi có thể truy cập vật lý thực sự đồng thời vào L , thì một quyền truy cập có thể dẫn đến nội dung không thể đoán trước của L . Hãy tưởng tượng điều gì sẽ xảy ra nội dung của L sau khi viết đồng thời 3 và 5 vào đó. Như vậy là hợp lý giả định rằng, cuối cùng, sự truy cập thực tế của li và lj tới L bằng cách nào đó, đang diễn ra nhanh chóng được tuần tự hóa (tuần tự hóa) bằng phần cứng để li và lj truy cập L lần lượt cái khác. Liệu việc tuần tự hóa tiềm ẩn như vậy có vô hiệu hóa được tất cả các mối nguy hiểm khi truy cập đồng thời không? Thật không may là không phải vậy. Nguyên nhân là do trật tự vật lý quyền truy cập của li và lj tới L là không thể đoán trước: sau khi xê-ri hóa, chúng ta không thể biết liệu li sẽ truy cập vật lý vào L trước hay sau lj . Do đó, tác động của các lệnh li và lj cũng không thể đoán trước được (Hình 2.3). Tại sao? Nếu cả P_i và P_j muốn đọc đồng thời từ L thì lệnh li và lj sẽ đọc cùng một nội dung của L , bất kể thứ tự tuần tự của chúng, vì vậy cả hai các đơn vị xử lý sẽ nhận được cùng nội dung của L —như mong đợi. Tuy nhiên, nếu một của các đơn vị xử lý muốn đọc từ L và đơn vị xử lý khác đồng thời muốn để ghi vào L thì dữ liệu mà bộ xử lý đọc nhận được sẽ phụ thuộc vào về việc lệnh đọc đã được đánh số thứ tự trước hay sau lệnh viết hướng dẫn. Hơn nữa, nếu cả P_i và P_j đồng thời cố gắng ghi vào L , thì nội dung thu được của L sẽ phụ thuộc vào cách li và lj được xê-ri hóa, tức là của li và lj là người cuối cùng viết thư cho L . Tóm lại, việc truy cập đồng thời vào cùng một vị trí có thể dẫn đến dữ liệu không thể đoán trước. trong các đơn vị xử lý truy cập cũng như ở vị trí được truy cập. Khi xem xét những phát hiện này, điều tự nhiên là đặt câu hỏi: Liệu tính không thể đoán trước này có làm cho Mô hình PRAM vô dụng? Câu trả lời là không, như chúng ta sẽ thấy ngay sau đây.

Các biến thể của PRAM

Các vấn đề trên đã khiến các nhà nghiên cứu xác định một số biến thể của PRAM khác nhau về

- (i) loại truy cập đồng thời nào vào cùng một vị trí được phép; và
- (ii) cách tránh tình trạng không thể đoán trước khi truy cập đồng thời vào cùng một vị trí.

Các biến thể được gọi là

- PRAM ghi độc quyền đọc (EREW-PRAM),
- PRAM ghi độc quyền đọc đồng thời (CREW-PRAM) và
- Đọc đồng thời Ghi đồng thời PRAM (CRCW-PRAM).

Bây giờ chúng tôi mô tả chúng chi tiết hơn:

• EREW-PRAM. Đây là biến thể thực tế nhất trong ba biến thể của PRAM mô hình. Mô hình EREW-PRAM không hỗ trợ truy cập đồng thời vào cùng một vị trí bộ nhớ; nếu một nỗ lực như vậy được thực hiện, mô hình sẽ ngừng thực thi chương trình của nó. Theo đó, giả định ngầm định là các chương trình chạy trên EREW-PRAM không bao giờ đưa ra các hướng dẫn có thể truy cập đồng thời vào cùng một vị trí; nghĩa là mọi quyền truy cập vào bất kỳ vị trí bộ nhớ nào đều phải độc quyền. Vì vậy việc xây dựng các chương trình như vậy là trách nhiệm của người thiết kế thuật toán.

• CREW-PRAM. Mô hình này hỗ trợ đọc đồng thời từ cùng một bộ nhớ location nhưng yêu cầu ghi độc quyền vào nó. Một lần nữa, gánh nặng xây dựng như vậy chương trình nằm trong trình thiết kế thuật toán.

• CRCW-PRAM. Đây là mô hình thực tế nhất của ba phiên bản phần mềm PRAM. Mô hình CRCW-PRAM cho phép đọc đồng thời từ cùng một bộ nhớ-hoạt động, ghi đồng thời vào cùng một vị trí bộ nhớ và đọc đồng thời từ và ghi vào cùng một vị trí bộ nhớ. Tuy nhiên, để tránh những điều không thể đoán trước hiệu ứng, các hạn chế bổ sung khác nhau được áp dụng cho việc ghi đồng thời. Cái này mang lại các phiên bản sau của mô hình CRCW-PRAM:

- ĐỐI TƯỢNG-CRCW-PRAM. Các đơn vị xử lý có thể đồng thời thử để ghi vào L, nhưng giả định rằng tất cả chúng đều cần ghi cùng một giá trị vào L. Tất nhiên, để đảm bảo điều đó là trách nhiệm của người thiết kế thuật toán.
- TUYỆT VỜI-CRCW-PRAM. Các đơn vị xử lý có thể đồng thời thử để ghi vào L (không nhất thiết phải có cùng giá trị), nhưng giả định rằng chỉ có một trong số họ sẽ thành công. Đơn vị xử lý nào sẽ thành công là điều không thể đoán trước được, vì vậy lập trình viên phải tính đến điều này khi thiết kế thuật toán.
- ƯU TIÊN-CRCW-PRAM. Có một thứ tự ưu tiên được áp dụng đối với đơn vị dừng; ví dụ: đơn vị xử lý có chỉ mục nhỏ hơn có mức độ ưu tiên cao hơn. Các đơn vị xử lý có thể đồng thời cố gắng ghi vào L, nhưng giả định rằng chỉ người có mức độ ưu tiên cao nhất mới thành công. Một lần nữa, người thiết kế thuật toán phải thấy trước và lưu ý mọi tình huống có thể xảy ra trong quá trình thực hiện.

– FUSION-CRCW-PRAM. Các đơn vị xử lý có thể đồng thời cố gắng viết thư cho L, nhưng người ta cho rằng

- ☐ đầu tiên một hoạt động cụ thể, được biểu thị bằng $\circ$, sẽ được áp dụng nhanh chóng cho tất cả các giá trị $v_1, v_2, \dots, v_k$ được ghi vào L, và
- ☐ chỉ khi đó kết quả $v_1 \circ v_2 \circ \dots \circ v_k$ của thao tác $\circ$ mới được ghi tới L

Phép toán $\circ$ được giả sử là có tính kết hợp và giao hoán, sao cho giá trị của biểu thức $v_1 \circ v_2 \circ \dots \circ v_k$ không phụ thuộc vào thứ tự thực hiện các hoạt động $\circ$. Ví dụ về phép tính $\circ$ là tổng (+), tích ($\cdot$), tối đa (tối đa), tối thiểu (tối thiểu), kết hợp logic ($\square$) và phân ly logic ($\square$).

☐ Sức mạnh tương đối của các biến thể

Vì những hạn chế về quyền truy cập đồng thời vào cùng một vị trí được nói lỏng khi chúng tôi chuyển từ EREW-PRAM sang CREW-PRAM rồi đến CRCW-PRAM, các biến thể của PRAM ngày càng trở nên ít thực tế hơn. Mặt khác, do những hạn chế bị loại bỏ, điều tự nhiên là có thể mong đợi rằng các biến thể có thể đạt được sức mạnh của chúng. Vì vậy chúng tôi đặt ra câu hỏi sau:

EREW-PRAM, CREW-PRAM và CRCW-PRAM có khác nhau về sức mạnh không?

Câu trả lời là có, nhưng không quá nhiều. Chuyển “quá nhiều” sương mù sẽ được làm rõ ở phần tiếp theo Định lý, trong đó CRCW-PRAM(p) biểu thị CRCW-PRAM với quá trình xử lý p các đơn vị và tương tự đối với EREW-PRAM(p). Một cách không chính thức, định lý cho chúng ta biết rằng bằng cách chuyển từ EREW-PRAM(p) sang CRCW-PRAM(p) “mạnh hơn” thời gian thực hiện song song của thuật toán song song có thể giảm theo một số yếu tố; tuy nhiên, hệ số này bị chặn ở trên và thực tế nó gần như ở cấp $O(\log p)$.

Định lý 2.1

Mọi thuật toán để giải bài toán tính toán Π trên CRCW-PRAM(p) nhanh hơn nhiều nhất là $O(\log p)$ lần so với thuật toán nhanh nhất cho giải Π trên EREW-PRAM(p).

Bằng chứng Ý tưởng Chúng tôi lần đầu tiên chứng minh rằng văn bản đồng thời của CONSISTENT-CRCW-PRAM(p)

Việc đưa vào cùng một vị trí có thể được thực hiện bằng các bước EREW-PRAM(p) theo $O(\log p)$. Do đó, EREW-PRAM có thể mô phỏng CRCW-PRAM, với hệ số làm chậm $O(\log p)$. Khi đó chúng ta chứng minh rằng hệ số làm chậm này là chặt chẽ, nghĩa là tồn tại một bài toán tính toán Π mà hệ số làm chậm thực sự là $(\log p)$. Như vậy Ví dụ, Π là bài toán tìm số lớn nhất của n số.

☐

Sự liên quan của mô hình PRAM

Chúng tôi đã giải thích tại sao mô hình PRAM lại không thực tế khi giả định một bộ nhớ chia sẻ không giới hạn, có thể định địa chỉ ngay lập tức. Điều này có nhất thiết có nghĩa

rằng mô hình PRAM không phù hợp cho mục đích triển khai thực tế của tính toán song song? Câu trả lời phụ thuộc vào những gì chúng ta mong đợi từ mô hình PRAM

hay nói chung hơn là cách chúng ta hiểu vai trò của lý thuyết.

Khi chúng ta cố gắng thiết kế một thuật toán để giải bài toán Π trên PRAM, chúng ta những nỗ lực có thể không kết thúc bằng một thuật toán thực tế, sẵn sàng để giải Π . Tuy nhiên,

thiết kế có thể tiết lộ điều gì đó vốn có của Π , cụ thể là Π có thể song song hóa được.

Nói cách khác, thiết kế có thể phát hiện trong Π các bài toán con, một số trong đó có thể, ít nhất về nguyên tắc, được giải quyết song song. Trong trường hợp này nó thường chứng minh rằng

các bài toán con thực sự có thể giải được song song theo cách tự do nhất (và không thực tế) PRAM, CRCW-PRAM.

Tại thời điểm này, tầm quan trọng của Định lý 2.1 trở nên rõ ràng: chúng ta có thể thay thế CRCW-PRAM bằng EREW-PRAM thực tế và giải Π ở phần sau. (Tất cả điều đó với cái giá là tốc độ giải quyết Π bị suy giảm có giới hạn).

Tóm lại, sự liên quan của PRAM được phản ánh trong phương pháp sau:

1. Thiết kế chương trình P giải Π trên mô hình CRCW-PRAM(p), trong đó p có thể phụ thuộc vào vấn đề Π . Lưu ý rằng thiết kế của P cho CRCW-PRAM là được mong đợi là dễ dàng hơn thiết kế cho EREW-PRAM, đơn giản vì CRCW-PRAM không có hạn chế truy cập đồng thời nào được tính đến.
2. Chạy P trên EREW-PRAM(p), được giả định là có thể mô phỏng đồng thời-ous truy cập vào cùng một vị trí.
3. Sử dụng Định lý 2.1 để đảm bảo rằng thời gian thực hiện song song của P trên EREW-PRAM(p) cao nhất là $O(\log p)$ -lần so với giá trị ít thực tế hơn CRCW-PRAM(p).

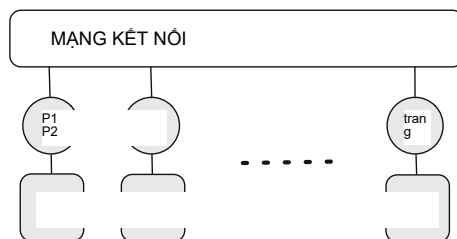
2.3.2 Máy bộ nhớ cục bộ

Mô hình LMM có p đơn vị xử lý, mỗi đơn vị có bộ nhớ cục bộ riêng (Hình 2.4).

Các đơn vị xử lý được kết nối với một mạng kết nối chung. Mỗi bộ xử lý có thể truy cập trực tiếp vào bộ nhớ cục bộ của nó. Ngược lại, nó có thể truy cập bộ nhớ không cục bộ (tức là bộ nhớ cục bộ của đơn vị xử lý khác) chỉ bằng cách gửi yêu cầu bộ nhớ thông qua mạng kết nối.

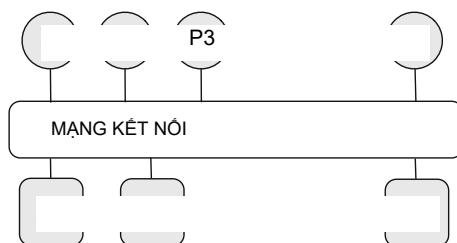
Giả định là tất cả các hoạt động cục bộ, bao gồm cả việc truy cập bộ nhớ cục bộ, lấy đơn vị thời gian. Ngược lại, thời gian cần thiết để truy cập bộ nhớ không cục bộ phụ thuộc vào trên

- khả năng của mạng kết nối và
- mô hình truy cập bộ nhớ không cục bộ trùng hợp của các đơn vị xử lý khác như các truy cập có thể làm tắc nghẽn mạng kết nối.



Hình 2.4 Mô hình LMM của tính toán song song có các đơn vị xử lý với bộ nhớ cục bộ của nó. Mỗi đơn vị xử lý truy cập trực tiếp vào bộ nhớ cục bộ của nó và có thể truy cập vào bộ nhớ cục bộ của đơn vị xử lý khác, bộ nhớ thông qua mạng kết nối

Hình 2.5 Mô hình MMM tính toán song song có đơn vị xử lý p và m mô-đun bộ nhớ. Mỗi đơn vị xử lý có thể truy cập bất kỳ mô-đun bộ nhớ nào thông qua mạng kết nối. Không có ký ức địa phương đến các đơn vị xử lý



2.3.3 Máy mô-đun bộ nhớ

Mô hình MMM (Hình 2.5) bao gồm p đơn vị xử lý và m mô-đun bộ nhớ mỗi trong số đó có thể được truy cập bởi bất kỳ đơn vị xử lý nào thông qua một kết nối chung mạng. Không có bộ nhớ cục bộ cho các đơn vị xử lý. Một đơn vị xử lý có thể truy cập mô-đun bộ nhớ bằng cách gửi yêu cầu bộ nhớ thông qua kết nối mạng.

Giả định rằng các đơn vị xử lý và mô-đun bộ nhớ được sắp xếp theo cách như vậy theo cách mà—khi không có truy cập trùng khớp—thời gian cho bất kỳ quá trình xử lý nào đơn vị truy cập vào bất kỳ mô-đun bộ nhớ nào gần như giống nhau. Tuy nhiên, khi có truy cập trùng khớp, thời gian truy cập phụ thuộc vào

- khả năng của mạng kết nối và
- kiểu truy cập bộ nhớ trùng hợp.

2.4 Tác động của truyền thông

Chúng ta đã thấy rằng cả mô hình LMM và mô hình MMM đều sử dụng kết nối liên mạng chuyển động để truyền các yêu cầu bộ nhớ tới các bộ nhớ không cục bộ (xem Hình.2.4 và 2.5). Trong phần này chúng ta tập trung vào vai trò của một mạng kết nối trong một

mô hình đa bộ xử lý và tác động của nó đến độ phức tạp thời gian song song của thuật toán.

2.4.1 Mạng kết nối

Kể từ buổi bình minh của điện toán song song, dấu hiệu chính của hệ thống song song đã là loại bộ xử lý trung tâm (CPU) và mạng kết nối làm việc. Điều này hiện đang thay đổi. Các thử nghiệm gần đây đã chỉ ra rằng thời gian thực hiện

của hầu hết các ứng dụng song song trong thế giới thực đang ngày càng phụ thuộc nhiều hơn vào

thời gian giao tiếp hơn là thời gian tính toán. Vì vậy, với số lượng

sự hợp tác của các đơn vị xử lý hoặc máy tính tăng lên, hiệu suất của các liên kết

mạng lưới kết nối đang trở nên quan trọng hơn hiệu suất của quá trình xử lý

đơn vị. Cụ thể, mạng lưới kết nối có ảnh hưởng lớn đến hiệu quả và

khả năng mở rộng của một máy tính song song trên hầu hết các ứng dụng song song trong thế giới thực. Ở nơi khác

Nói cách khác, hiệu suất cao của mạng kết nối cuối cùng có thể phản ánh

tốc độ cao hơn, bởi vì mạng kết nối như vậy có thể rút ngắn tổng thể

thời gian thực hiện song song cũng như tăng số lượng đơn vị xử lý có thể

được khai thác có hiệu quả.

Hiệu suất của một mạng kết nối phụ thuộc vào một số yếu tố. Ba

quan trọng nhất là định tuyến, thuật toán điều khiển luồng và mạng

cấu trúc liên kết. Định tuyến ở đây là quá trình chọn đường dẫn cho lưu lượng trong một kết nối

mạng lưới hoạt động; Kiểm soát luồng là quá trình quản lý tốc độ truyền dữ liệu

giữa hai nút để ngăn chặn người gửi nhanh áp đảo người nhận chậm; và

Cấu trúc liên kết mạng là sự sắp xếp của các thành phần khác nhau, chẳng hạn như giao tiếp

các nút và kênh của mạng kết nối.

Đối với các thuật toán định tuyến và điều khiển luồng, các kỹ thuật hiệu quả đã được biết đến.

và được sử dụng. Ngược lại, cấu trúc liên kết mạng chưa điều chỉnh theo những thay đổi về công nghệ.

xu hướng công nghệ nhanh chóng như các thuật toán định tuyến và điều khiển luồng. Đây là một

lý do mà nhiều cấu trúc liên kết mạng được phát hiện ngay sau khi ra đời

tính toán song song vẫn đang được sử dụng rộng rãi. Một lý do khác là sự tự do

người dùng cuối có thể lựa chọn cấu trúc liên kết mạng thích hợp cho

mức sử dụng dự kiến. (Do các tiêu chuẩn hiện đại, không có sự tự do lựa chọn như vậy

hoặc thay đổi thuật toán định tuyến hoặc điều khiển luồng). Kết quả là, một bước tiến xa hơn trong

việc tăng hiệu suất có thể được mong đợi đến từ những cải tiến trong topol-

ogy của các mạng kết nối. Ví dụ, những cải tiến như vậy sẽ cho phép

mạng kết nối để thích ứng linh hoạt với ứng dụng hiện tại trong một số

cách tối ưu.

2.4.2 Thuộc tính cơ bản của mạng kết nối

Chúng ta có thể phân loại các mạng kết nối theo nhiều cách và mô tả chúng theo các thông số khác nhau. Để xác định hầu hết các tham số này, lý thuyết đồ thị là phương pháp tốt nhất khung toán học thanh lịch. Cụ thể hơn, một mạng kết nối có thể

được mô hình hóa dưới dạng đồ thị $G(N, C)$, trong đó N là tập hợp các nút truyền thông và C là một tập hợp các liên kết truyền thông (hoặc các kênh) giữa các nút truyền thông. Dựa trên quan điểm lý thuyết đồ thị về các mạng kết nối này, chúng ta có thể định nghĩa các tham số đại diện cho cả thuộc tính tô pô và thuộc tính hiệu suất của các mạng kết nối. Hãy để chúng tôi mô tả cả hai loại tài sản.

Thuộc tính cấu trúc liên kết của mạng kết nối

Các thuộc tính tô pô quan trọng nhất của các mạng kết nối, được xác định bởi các khái niệm lý thuyết đồ thị là

- mức độ nút,
- đều đặn,
- tính đối xứng,
- đường kính,
- sự đa dạng của đường đi, và
- khả năng mở rộng quy mô.

Sau đây chúng tôi định từng yếu tố đó và đưa ra nhận xét khi thích hợp:

- Mức độ nút là số d kênh mà thông qua đó một nút tìon được kết nối với các nút truyền thông khác. Lưu ý, mức độ nút đó chỉ bao gồm các cổng dành cho giao tiếp mạng, mặc dù một cổng giao tiếp nút tìon cũng cần các cổng để kết nối với (các) phần tử xử lý và cổng cho các kênh dịch vụ hoặc bảo trì.
- Một mạng kết nối được gọi là thường xuyên nếu tất cả các nút truyền thông có cùng mức độ nút; nghĩa là tồn tại $a, d > 0$ sao cho mọi giao tiếp nút có mức độ nút d .
- Một mạng kết nối được gọi là đối xứng nếu tất cả các nút truyền thông sở hữu "cùng quan điểm" về mạng; tức là có sự đồng hình ánh xạ bất kỳ nút giao tiếp nào tới bất kỳ nút giao tiếp nào khác. Ở dạng đối xứng mạng kết nối, tài có thể được phân bổ đều qua tất cả các mạng cộng đồng các nút cation, do đó làm giảm vấn đề tắc nghẽn. Nhiều triển khai thực tế của các mạng kết nối dựa trên các đồ thị chính quy đối xứng vì chúng các đặc tính tô pô hiệu quả dẫn đến việc định tuyến đơn giản và cân bằng tải hợp lý dưới lưu lượng thông nhất.
- Để di chuyển từ nút nguồn đến nút đích, một gói phải đi qua thông qua một loạt các phần tử, chẳng hạn như bộ định tuyến hoặc bộ chuyển mạch, cùng nhau tạo thành một đường dẫn (hoặc tuyến đường) giữa nút nguồn và nút đích. Số lượng các nút truyền thông mà gói tin đi qua dọc theo đường dẫn này được gọi là hop đếm. Trong trường hợp tốt nhất, hai nút giao tiếp thông qua đường dẫn có số bước nhảy tối thiểu, 1, được thực hiện trên tất cả các đường dẫn giữa hai nút. Vì tôi có thể thay đổi tùy theo nút nguồn và nút đích, chúng tôi cũng sử dụng khoảng cách trung bình, l_{avg} , là l trung bình được lấy trên tất cả các cặp nút có thể có. Một điều quan trọng Đặc điểm của bất kỳ cấu trúc liên kết nào là đường kính, l_{max} , là giá trị lớn nhất của tất cả số bước nhảy tối thiểu, được thực hiện trên tất cả các cặp nút nguồn và đích.

• Trong một mạng kết nối có thể tồn tại nhiều đường dẫn giữa hai nút. Trong trường hợp như vậy, các nút có thể được kết nối theo nhiều cách. Một gói bắt đầu từ nguồn nút sẽ có nhiều tuyến đường để đến nút đích. các gói có thể đi theo các tuyến khác nhau (hoặc thậm chí là các phần tiếp theo khác nhau của một phần được truyền qua) của một tuyến đường) tùy thuộc vào tình hình hiện tại trong mạng. Một sự kết nối mạng có tính đa dạng đường dẫn cao cung cấp nhiều lựa chọn thay thế hơn khi các gói cần để tìm kiếm điểm đến và/hoặc tránh chướng ngại vật.

• Khả năng mở rộng là (i) khả năng của hệ thống để xử lý khối lượng công việc ngày càng tăng, hoặc (ii) tiềm năng của hệ thống sẽ được mở rộng để đáp ứng sự tăng trưởng đó. các khả năng mở rộng là quan trọng ở mọi cấp độ. Ví dụ, khối xây dựng cơ bản phải có thể dễ dàng kết nối với các khối khác một cách thống nhất. Hơn nữa, cùng một tòa nhà khối phải được sử dụng để xây dựng các mạng kết nối có kích thước khác nhau, chỉ với sự suy giảm hiệu năng nhỏ đối với máy tính song song có kích thước tối đa. liên-mạng kết nối có tác động quan trọng đến khả năng mở rộng của các máy tính song song dựa trên mô hình đa bộ xử lý LMM hoặc MMM. Để đánh giá cao điều đó, lưu ý rằng khả năng mở rộng bị hạn chế nếu mức độ nút được cố định.

Thuộc tính hiệu suất của mạng kết nối

Các đặc tính hiệu suất chính của các mạng kết nối là

- băng thông kênh,
- băng thông chia đôi, và
- độ trễ.

Bây giờ chúng tôi xác định từng người trong số họ và đưa ra nhận xét khi thích hợp:

• Băng thông kênh, trong băng thông ngắn, là lượng dữ liệu được hoặc theo- về mặt trực quan có thể được truyền đạt qua một kênh trong một khoảng thời gian nhất định. Trong hầu hết các trường hợp, băng thông kênh có thể được xác định đầy đủ bằng cách sử dụng một mô hình giao tiếp đơn giản ủng hộ rằng thời gian giao tiếp t_{comm} , cần thiết để truyền dữ liệu đã cho qua kênh, là tổng $t_s + t_d$ thời gian khởi động t_s , cần thiết để thiết lập phần mềm và phần cứng của kênh, và thời gian truyền dữ liệu t_d , trong đó $t_d = m \cdot t_w$, tích của số từ tạo dữ liệu, m và thời gian truyền trên mỗi từ, t_w . Sau đó kênh băng thông là $1/t_w$.

• Một mạng kết nối nhất định có thể được cắt thành hai (gần như) thành phần có kích thước bằng nhau

Nent. Nói chung, điều này có thể được thực hiện bằng nhiều cách. Đưa ra một sự cắt giảm kết nối mạng, băng thông cắt là tổng băng thông kênh của tất cả các kênh mới hai thành phần. Băng thông cắt nhỏ nhất được gọi là chia đôi băng thông (BBW) của mạng kết nối. Đường cắt tương ứng là trường hợp xấu nhất là cắt mạng kết nối. Thỉnh thoảng, dải phân đôi- cần có chiều rộng trên mỗi nút (BBWN); chúng tôi định nghĩa nó là BBW chia cho $|N|$, số lượng nút trong mạng. Tất nhiên, cả BBW và BBWN đều phụ thuộc vào topo của mạng và băng thông kênh. Nói chung là ngày càng tăng

băng thông của mạng kết nối có thể có những tác động có lợi như tăng xung nhịp CPU (nhớ lại Mục 2.4.1).

- Độ trễ là thời gian cần thiết để một gói tin di chuyển từ nút nguồn tới nút đích. Nhiều ứng dụng, đặc biệt là những ứng dụng sử dụng tin nhắn ngắn, nhạy cảm với độ trễ theo nghĩa là hiệu quả của các ứng dụng này phụ thuộc rất nhiều về độ trễ. Đối với các ứng dụng như vậy, chi phí phần mềm của chúng có thể trở thành một vấn đề lớn

yếu tố ảnh hưởng đến độ trễ. Cuối cùng, độ trễ được giới hạn dưới thời gian trong đó ánh sáng truyền qua khoảng cách vật lý giữa hai nút.

Việc truyền dữ liệu từ nút nguồn đến nút đích được đo bằng thuật ngữ của nhiều đơn vị khác nhau được xác định như sau:

- gói, lượng dữ liệu nhỏ nhất có thể được truyền bằng phần cứng,
- FLIT (chữ số điều khiển luồng), lượng dữ liệu được sử dụng để phân bổ không gian bộ đệm trong một số kỹ thuật kiểm soát dòng chảy;
- PHIT (chữ số vật lý), lượng dữ liệu có thể được truyền trong một chu kỳ.

Các đơn vị này có liên quan chặt chẽ đến băng thông và độ trễ của mạng.

Ánh xạ các mạng kết nối vào không gian thực

Một mạng kết nối của bất kỳ cấu trúc liên kết nhất định nào, thậm chí nếu được định nghĩa một cách trừu tượng

không gian chiều cao hơn, cuối cùng phải được ánh xạ vào không gian vật lý, ba chiều không gian chiều (3D). Điều này có nghĩa là tất cả các con chip và băng mạch in tạo nên mạng kết nối phải được phân bổ địa điểm vật lý.

Thật không may, đây không phải là một nhiệm vụ tầm thường. Lý do là việc lập bản đồ thường có

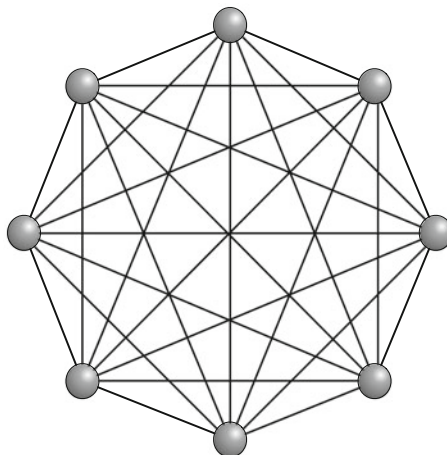
để tối ưu hóa các tiêu chí nhất định, thường mâu thuẫn, đồng thời tôn trọng hạn chế khác nhau. Dưới đây là một số ví dụ:

- Một hạn chế như vậy là số lượng chân I/O trên mỗi chip hoặc trên mỗi bản in băng mạch được giới hạn ở trên. Một tiêu chí tối ưu hóa thông thường là, để ngăn chặn việc giảm tốc độ dữ liệu, cấp càng ngắn càng tốt. Nhưng do kích thước đáng kể của các thành phần phần cứng và do những hạn chế vật lý của 3D-không gian, ánh xạ có thể kéo dài đáng kể các đường dẫn nhất định, tức là các nút ở gần trong không gian nhiều chiều hơn có thể được ánh xạ tới các vị trí ở xa trong không gian 3D.
- Chúng ta có thể muốn lập bản đồ các đơn vị xử lý có khả năng giao tiếp mạnh mẽ càng gần nhau
- với nhau nhất có thể, lý tưởng nhất là trên cùng một con chip. Bằng cách này chúng ta có thể giảm thiểu
- tác động của giao tiếp. Thật không may, việc xây dựng tối ưu như vậy
- ánh xạ là vấn đề tối ưu hóa NP-hard.
- Một tiêu chí bổ sung có thể là mức tiêu thụ điện năng được giảm thiểu.

2.4.3 Phân loại mạng kết nối

Mạng kết nối có thể được phân loại thành mạng trực tiếp và mạng gián tiếp. đây là đặc điểm chính của từng loại.

Hình.2.6 A được kết nối đầy đủ
mạng có $n = 8$ nút



Mạng trực tiếp

Một mạng được gọi là trực tiếp khi mỗi nút được kết nối trực tiếp với các nút lân cận của nó. Một nút có thể có bao nhiêu hàng xóm? Trong một mạng được kết nối đầy đủ, mỗi $n = |N|$ các nút được kết nối trực tiếp với tất cả các nút khác, vì vậy mỗi nút có $n - 1$ hàng xóm. (Xem hình 2.6).

Vì mạng như vậy có 1

$2n(n - 1) = (n2)$ kết nối trực tiếp, nó chỉ có thể là

được sử dụng để xây dựng các hệ thống có số lượng nút nhỏ n . Khi n lớn, mỗi nút được kết nối trực tiếp với một tập hợp con thích hợp của các nút khác, trong khi giao tiếp tới các nút còn lại đạt được bằng cách định tuyến tin nhắn thông qua các nút trung gian.

Một ví dụ về mạng kết nối trực tiếp như vậy là hypercube; xem hình.2.13 trên trang 28.

Mạng gián tiếp

Một mạng gián tiếp kết nối các nút thông qua các thiết bị chuyển mạch. Thông thường, nó kết nối pro-

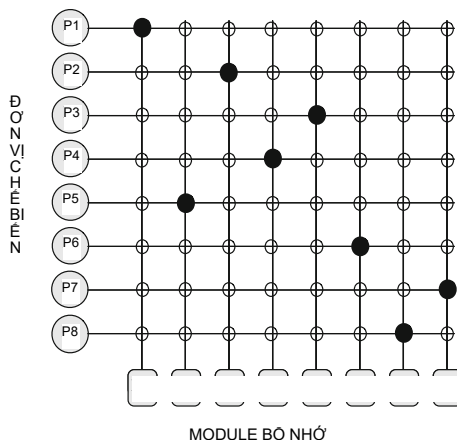
các thiết bị dừng ở một đầu của mạng và các mô-đun bộ nhớ ở đầu kia của mạng. Mạch đơn giản nhất để kết nối các bộ xử lý với các mô-đun bộ nhớ là công tắc thanh ngang được kết nối đầy đủ (Hình 2.7). Ưu điểm của nó là có thể thiết lập kết nối giữa các đơn vị xử lý và mô-đun bộ nhớ một cách tùy ý.

Tại mỗi giao điểm của đường ngang và đường dọc đều có một điểm giao nhau. Một điểm giao nhau

là một công tắc nhỏ có thể mở bằng điện ($\circ$) hoặc đóng ($\wedge$), tùy thuộc vào liệu các đường ngang và dọc có được kết nối hay không. Trong hình.2.7 chúng tôi thấy tám điểm giao nhau được đóng đồng thời, cho phép kết nối giữa các cặp $(P1, M1)$, $(P2, M3)$, $(P3, M5)$, $(P4, M4)$, $(P5, M2)$, $(P6, M6)$, $(P7, M8)$ và $(P8, M7)$ tại cùng một lúc. Nhiều sự kết hợp khác cũng có thể.

Thật không may, thanh ngang được kết nối đầy đủ có độ phức tạp quá lớn để sử dụng để kết nối số lượng lớn cổng đầu vào và đầu ra. Cụ thể, số lượng điểm giao nhau tăng theo chiều, trong đó p và m là số lượng đơn vị xử lý và mô-đun bộ nhớ tương ứng. Với $p = m = 1000$ con số này tương đương với một triệu chéo những điểm không khả thi. (Tuy nhiên, đối với các hệ thống cỡ trung bình, thanh ngang

Hình.2.7 A được kết nối đầy đủ công tắc thanh ngang kết nối 8 nút đến 8 nút



thiết kế có thể thực hiện được và các công tắc thanh ngang nhỏ được kết nối đầy đủ được sử dụng làm cơ bản các khối xây dựng trong các bộ chuyển mạch và bộ định tuyến lớn hơn). Đây là lý do tại sao các mạng gián tiếp kết nối các nút thông qua nhiều thiết bị chuyển mạch. các Bản thân các thiết bị chuyển mạch thường được kết nối với nhau theo từng giai đoạn bằng cách sử dụng một mô hình kết nối giữa các giai đoạn. Các mạng gián tiếp như vậy được gọi là mạng đa mạng kết nối giai đoạn; chúng tôi sẽ mô tả chúng chi tiết hơn ở trang 29.

Mạng gián tiếp có thể được phân loại thêm như sau:

- Mạng không chặn có thể kết nối bất kỳ nguồn nhận rỗi nào với bất kỳ đích nhận rỗi nào, bất kể các kết nối đã được thiết lập trên mạng. Điều này là do với cấu trúc liên kết mạng để đảm bảo sự tồn tại của nhiều đường dẫn giữa nguồn và đích.
- Việc sắp xếp lại các mạng có thể sắp xếp lại có thể sắp xếp lại các kết nối có đã được thiết lập trên mạng theo cách mà một kết nối mới có thể được thành lập. Một mạng như vậy có thể thiết lập tất cả các kết nối có thể có giữa đầu vào và đầu ra.
- Trong mạng chặn, kết nối đã được thiết lập trên mạng có thể ngăn cản việc thiết lập một kết nối mới giữa nguồn và đích quốc gia, ngay cả khi nguồn và đích đều miễn phí. Một mạng như vậy không thể luôn cung cấp kết nối giữa nguồn và đích miễn phí tùy ý.

Sự khác biệt giữa mạng trực tiếp và gián tiếp ngày nay ít rõ ràng hơn. Mỗi mạng trực tiếp có thể được biểu diễn dưới dạng mạng gián tiếp vì mọi nút trong mạng trực tiếp có thể được biểu diễn dưới dạng bộ định tuyến với phần tử xử lý riêng được kết nối tới các bộ định tuyến khác. Tuy nhiên, đối với cả mạng kết nối trực tiếp và gián tiếp, thanh ngang đầy đủ, như một công tắc lý tưởng, là trung tâm của truyền thống.

2.4.4 Cấu trúc liên kết của mạng kết nối

Không khó để nhận thấy rằng tồn tại nhiều cấu trúc liên kết mạng có khả năng liên kết kết nối các đơn vị xử lý và m mô-đun bộ nhớ (xem Bài tập). Tuy nhiên, không mọi cấu trúc liên kết mạng đều có khả năng truyền tải các yêu cầu bộ nhớ đủ nhanh để sao lưu hiệu quả tính toán song song. Hơn nữa, hóa ra mạng Cấu trúc liên kết có ảnh hưởng lớn đến hiệu suất của mạng kết nối và do đó, tính toán song song. Ngoài ra, cấu trúc liên kết mạng có thể phát sinh những khó khăn đáng kể trong việc xây dựng mạng thực tế và chi phí của nó. Trong vài thập kỷ qua, các nhà nghiên cứu đã đề xuất, phân tích, xây dựng, thử nghiệm, và sử dụng nhiều cấu trúc liên kết mạng khác nhau. Bây giờ chúng tôi đưa ra một cái nhìn tổng quan về những điều đáng chú ý nhất hoặc phổ biến: bus, lưới, lưới 3D, hình xuyên, siêu khối, đa tầng mạng và cây bèo.

xe buýt

Đây là cấu trúc liên kết mạng đơn giản nhất. Xem hình 2.8. Nó có thể được sử dụng ở cả địa phương-

máy bộ nhớ (LMM) và máy mô-đun bộ nhớ (MMM). Trong cả hai trường hợp, tất cả các bộ xử lý và mô-đun bộ nhớ được kết nối với một bus duy nhất. Trong mỗi bước, nhiều nhất một phần dữ liệu có thể được ghi lên xe buýt. Đây có thể là một yêu cầu từ một đơn vị xử lý để đọc hoặc ghi một giá trị bộ nhớ hoặc nó có thể là phản hồi từ bộ xử lý hoặc mô-đun bộ nhớ chứa giá trị.

Khi trong máy mô-đun bộ nhớ, bộ xử lý muốn đọc bộ nhớ

word, trước tiên nó phải kiểm tra xem xe buýt có bận không. Nếu bus không hoạt động, bộ xử lý

đặt địa chỉ của từ mong muốn trên bus, đưa ra các tín hiệu điều khiển cần thiết,

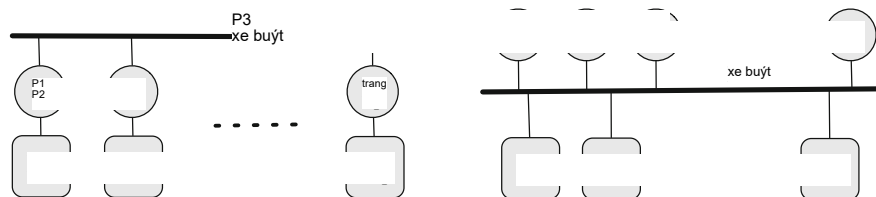
và đợi cho đến khi bộ nhớ đặt từ mong muốn lên xe buýt. Tuy nhiên, nếu xe buýt

bận khi một đơn vị xử lý muốn đọc hoặc ghi bộ nhớ, đơn vị xử lý

phải đợi cho đến khi xe buýt ngừng hoạt động. Đây là điểm hạn chế của cấu trúc liên kết xe buýt

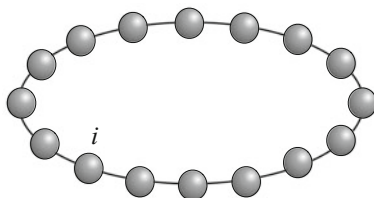
trở nên rõ ràng. Nếu có một số lượng nhỏ các đơn vị xử lý, chẳng hạn như hai hoặc ba, sự tranh chấp về xe buýt có thể quản lý được; nhưng với số lượng đơn vị xử lý lớn hơn, ví dụ 32, sự tranh chấp trở nên không thể chịu nổi vì hầu hết các đơn vị xử lý sẽ chờ đợi hầu hết thời gian.

Để giải quyết vấn đề này, chúng tôi thêm bộ đệm cục bộ vào từng đơn vị xử lý. Bộ đệm có thể được đặt trên bo mạch bộ xử lý, bên cạnh chip bộ xử lý, bên trong chip đơn vị xử lý hoặc sự kết hợp nào đó của cả ba. Nói chung, bộ nhớ đệm là không được thực hiện trên cơ sở từ riêng lẻ mà trên cơ sở các khối bao gồm, chẳng hạn, 64 byte. Khi một từ được tham chiếu bởi một đơn vị xử lý, toàn bộ khối của từ đó



Hình 2.8 Bus là cấu trúc liên kết mạng đơn giản nhất

Hình.2.9 Một chiếc nhẫn. Mỗi nút đại diện cho một đơn vị xử lý với bộ nhớ cục bộ



được lấy vào bộ đệm cục bộ của đơn vị xử lý. Sau đó có thể đọc nhiều lần được đáp ứng từ bộ nhớ đệm cục bộ. Kết quả là sẽ có ít lưu lượng xe buýt hơn và hệ thống sẽ có thể hỗ trợ nhiều đơn vị xử lý hơn. Chúng ta thấy rằng lợi ích thực tế của việc sử dụng xe buýt là (i) chúng đơn giản để xây dựng và (ii) tương đối dễ dàng để phát triển các giao thức cho phép các đơn vị xử lý để lưu trữ cục bộ các giá trị bộ nhớ (vì tất cả các đơn vị xử lý và mô-đun bộ nhớ có thể quan sát giao thông trên xe buýt). Nhược điểm rõ ràng của việc sử dụng xe buýt là các đơn vị xử lý phải thay phiên nhau truy cập vào bus. Điều này hàm ý rằng càng nhiều các đơn vị xử lý được thêm vào bus, thời gian trung bình để thực hiện truy cập bộ nhớ tăng tỷ lệ thuận với số lượng đơn vị xử lý.

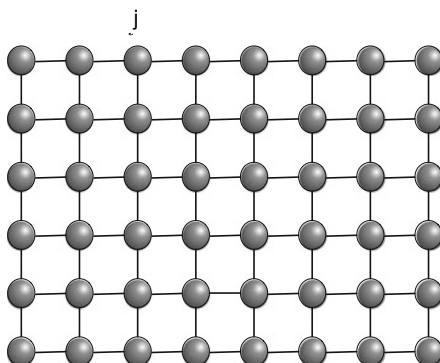
Chiếc nhẫn

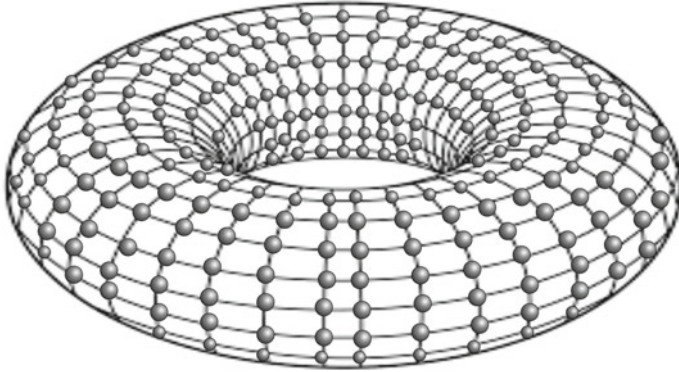
Ring là một trong những mạng kết nối đơn giản nhất và lâu đời nhất. Cho n các nút, chúng được sắp xếp theo kiểu tuyến tính sao cho mỗi nút có một nhãn i riêng biệt, trong đó $0 \leq i \leq n-1$. Mỗi nút được kết nối với hai nút lân cận, một ở bên trái và một bên phải. Do đó, nút có nhãn i được kết nối với nút có nhãn $i+1 \bmod n$ và $i-1 \bmod n$ (xem Hình.2.9). Vòng được sử dụng trong các máy bộ nhớ cục bộ (LMM).

Lưới 2D

Lưới hai chiều là mạng kết nối có thể được sắp xếp theo kiểu hình chữ nhật, sao cho mỗi công tắc trong lưới có một nhãn riêng biệt (i, j) , trong đó $0 \leq i \leq X-1$ và $0 \leq j \leq Y-1$. (Xem hình.2.10). Các giá trị X và Y xác định chiều dài các cạnh của lưới. Do đó, số lượng công tắc trong lưới là XY . Mỗi switch, ngoại trừ những switch ở hai bên của lưới, được kết nối với sáu switch lân cận: một ở phía bắc, một ở phía nam, một ở phía đông và một ở phía tây. Vì vậy một công tắc

Hình.2.10 Lưới 2D. Mỗi nút đại diện cho một bộ xử lý đơn vị có bộ nhớ cục bộ





Hình.2.11 Một hình xuyên 2D. Mỗi nút đại diện cho một đơn vị xử lý có bộ nhớ cục bộ

có nhãn (i, j) , trong đó $0 < i < X - 1$ và $0 < j < Y - 1$, được kết nối với các công tắc được dán nhãn $(i, j + 1)$, $(i, j - 1)$, $(i + 1, j)$ và $(i - 1, j)$.

Các mắt lưới thường xuất hiện trong các máy bộ nhớ cục bộ (LMM): một bộ xử lý (cùng với bộ nhớ cục bộ của nó) được kết nối với mỗi switch, do đó bộ nhớ từ xa truy cập được thực hiện bằng cách định tuyến tin nhắn qua mạng.

2D-Torus (Lưới 2D hình xuyên)

Trong lưới 2D, các công tắc ở hai bên không có kết nối nào với các công tắc bên các mặt đối lập. Mạng kết nối bù đắp cho điều này được gọi là lưới hình xuyên hoặc chỉ hình xuyên khi $d = 2$. (Xem hình.2.11). Vì vậy, trong hình xuyên mọi công tắc đặt tại (i, j) được nối với bốn công tắc khác đặt tại $(i, j + 1 \bmod Y)$, $(i, j - 1 \bmod Y)$, $(i + 1 \bmod X, j)$ và $(i - 1 \bmod X, j)$.

Các hình xuyên xuất hiện trong các máy bộ nhớ cục bộ (LMM): mỗi switch được kết nối một đơn vị xử lý với bộ nhớ cục bộ của nó. Mỗi đơn vị xử lý có thể truy cập bất kỳ điều khiển từ xa nào

bộ nhớ bằng cách định tuyến các thông điệp qua hình xuyên.

Lưới 3D và Hình xuyên 3D

Lưới ba chiều tương tự như lưới hai chiều. (Xem hình.2.12). Bây giờ mỗi công tắc trong lưới có nhãn riêng biệt (i, j, k) , trong đó $0 \leq i \leq X - 1$, $0 \leq j \leq Y - 1$, và $0 \leq k \leq Z - 1$. Các giá trị X , Y và Z xác định độ dài các cạnh của

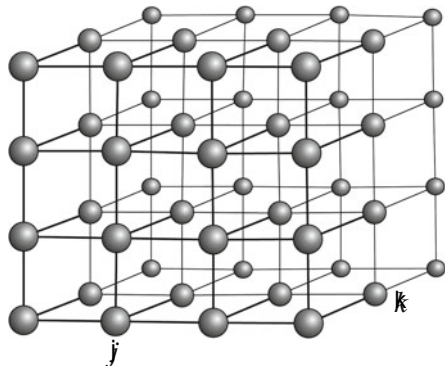
lưới, vậy số lượng công tắc trong đó là XYZ . Mọi công tắc, ngoại trừ những công tắc trên các cạnh của lưới, hiện được kết nối với sáu hàng xóm: một ở phía bắc, một ở phía bắc phía nam, một phía đông, một phía tây, một phía trên và một phía dưới. Vì vậy, một công tắc có nhãn

(i, j, k) , trong đó $0 < i < X - 1$, $0 < j < Y - 1$ và $0 < k < Z - 1$, được kết nối với các công tắc $(i, j + 1, k)$, $(i, j - 1, k)$, $(i + 1, j, k)$, $(i - 1, j, k)$, $(i, j, k + 1)$ và $(i, j, k - 1)$. Những mắt lưới như vậy thường xuất hiện trong LMM.

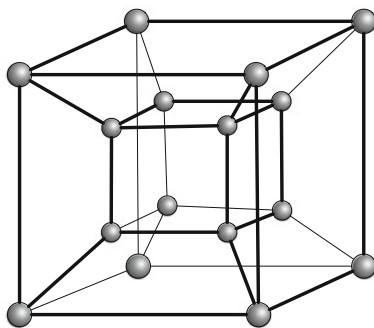
Chúng ta có thể mở rộng lưới 3D thành lưới 3D hình xuyên bằng cách thêm các cạnh kết nối các nút nằm ở phía đối diện của lưới 3D. (Hình ảnh bị bỏ qua). Một công tắc có nhãn (i, j, k) được nối với các công tắc $(i + 1 \bmod X, j, k)$, $(i - 1 \bmod X, j, k)$, $(i, j + 1 \bmod Y, k)$, $(i, j - 1 \bmod Y, k)$, $(i, j, k + 1 \bmod Z)$ và $(i, j, k - 1 \bmod Z)$.

Lưới 3D và lưới 3D hình xuyên được sử dụng trong các máy bộ nhớ cục bộ (LMM).

Hình.2.12 Lưới 3D. Mỗi nút đại diện cho một bộ xử lý đơn vị có bộ nhớ cục bộ



Hình.2.13 Một siêu khối. Mỗi nút đại diện cho một bộ xử lý cục bộ trí nhớ



siêu khối

Hypercube là một mạng kết nối có $n = 2^b$ nút, với một số $b \geq 0$.

(Xem hình 2.13). Mỗi nút có một nhãn riêng biệt bao gồm b bit. Hai nút là được kết nối bằng một liên kết truyền thông nếu chỉ khi nhãn của chúng khác nhau một cách chính xác

vị trí bit. Do đó, mỗi nút của siêu khối có $b = \log_2 n$ lân cận.

Hypercube được sử dụng trong các máy bộ nhớ cục bộ (LMM).

□ Họ cấu trúc liên kết mạng k -ary d -Cube

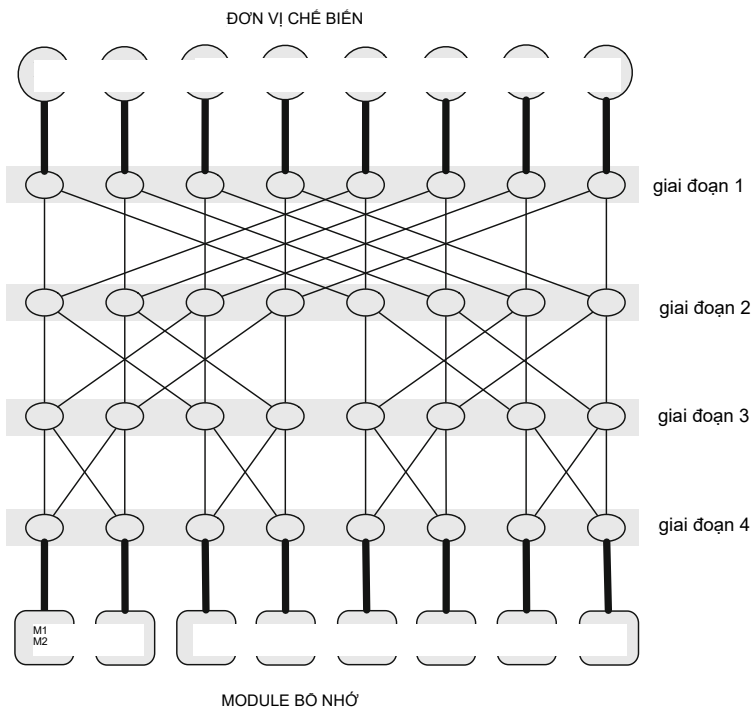
Điều thú vị là chiếc nhẫn, hình xuyên 2D, hình xuyên 3D, siêu khối và nhiều thứ khác tất cả các cấu trúc liên kết đều thuộc về một họ lớn hơn của cấu trúc liên kết k -ary d -cube.

Cho $k \geq 1$ và $d \geq 1$, cấu trúc liên kết k -ary d -cube là một họ của một số "dạng lưới" nhất định các cấu trúc liên kết có chung kiểu dáng mà chúng được xây dựng. Nói cách khác, sự

Cấu trúc liên kết k -ary d -cube là sự tổng quát hóa của một số cấu trúc liên kết nhất định.

Tham số d là được gọi là kích thước của các cấu trúc liên kết này và k là độ dài cạnh của chúng, số lượng các nút dọc theo mỗi hướng d . Thời trạng trong đó cấu trúc liên kết k -ary d -cube được xây dựng được xác định theo kiểu quy nạp (trên chiều d):

Một khối k -ary d được xây dựng từ k khối k -ary $(d-1)$ khác bằng cách kết nối các nút có vị trí giống nhau thành các vòng.



Hình 2.14 Mạng kết nối 4 giai đoạn có khả năng kết nối 8 bộ xử lý với 8 bộ nhớ mô-đun. Mỗi công tắc có thể thiết lập kết nối giữa cặp đầu vào và đầu ra tùy ý kênh

Định nghĩa quy nạp này cho phép chúng ta xây dựng một cách có hệ thống k-ary d-cube thực tế

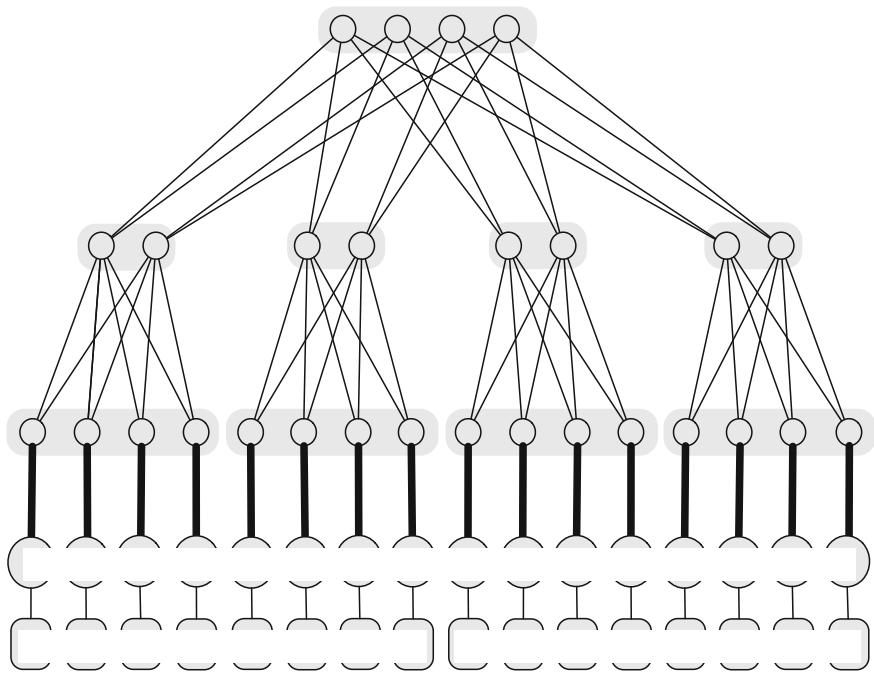
cấu trúc liên kết và phân tích các đặc tính cấu trúc liên kết và hiệu suất của chúng. Ví dụ, chúng ta có thể suy ra rằng cấu trúc liên kết k-ary d-cube chứa $n = kd$ nút truyền thông và $c = dn = dkd$ liên kết truyền thông, trong khi đường kính là $l_{max} = dk$

2 và khoảng cách trung bình giữa hai nút là $l_{avg} = l_{max} \cdot \frac{4 - 1}{k}$ (nếu k chẵn) hoặc $l_{avg} = d \cdot \frac{4 - 1}{k}$ (nếu k lẻ). Thật không may, mặc dù có cấu trúc đệ quy đơn giản, các khối k-ary d có khả năng mở rộng mở rộng kém.

Mạng đa tầng

Mạng nhiều tầng kết nối một bộ chuyển mạch, gọi là chuyển mạch đầu vào, với một bộ khác, gọi là công tắc đầu ra. Mạng đạt được điều này thông qua một chuỗi gồm nhiều giai đoạn, trong đó mỗi giai đoạn bao gồm các thiết bị chuyển mạch. (Xem hình 2.14). Đặc biệt, các công tắc đầu vào tạo thành giai đoạn đầu tiên và các công tắc đầu ra tạo thành giai đoạn cuối cùng.

các số d của các giai đoạn được gọi là độ sâu của mạng nhiều tầng. Thông thường, nhiều giai đoạn mạng cho phép gửi một phần dữ liệu từ bất kỳ công tắc đầu vào nào đến bất kỳ công tắc đầu ra nào. Việc này được thực hiện dọc theo một đường đi qua tất cả các giai đoạn của mạng theo thứ tự từ 1 đến d. Có nhiều cấu trúc liên kết mạng nhiều tầng khác nhau.



ĐƠN VỊ XỬ LÝ CÓ BỘ NHỚ ĐỊA PHƯƠNG

Hình 2.15 Cây béo. Mỗi switch có thể thiết lập kết nối giữa cặp sự cố tùy ý kênh

Mạng nhiều tầng thường được sử dụng trong các máy mô-đun bộ nhớ (MMM); ở đó, các bộ xử lý được gắn vào các cổng tắc đầu vào và các mô-đun bộ nhớ được gắn vào các cổng tắc đầu ra.

Cây Mỡ

Cây béo là một mạng có cấu trúc dựa trên cấu trúc của cây. (Xem hình 2.15).

Tuy nhiên, trái ngược với cây thông thường có các cạnh có cùng độ dày, ở dạng mập cây, các cạnh gần gốc cây sẽ "béo" (dày hơn) so với các cạnh gần gốc cây hơn.

đang ở phía dưới cây. Ý tưởng là mỗi nút của cây béo có thể đại diện cho nhiều chuyên mạch mạng và mỗi cạnh có thể đại diện cho nhiều kênh truyền thông. các Một cạnh đại diện cho nhiều kênh hơn thì dung lượng của nó càng lớn và cạnh đó càng béo. Vì vậy, năng lực của các cạnh gần gốc cây béo lớn hơn nhiều so với năng lực của các cạnh gần lá.

Cây béo có thể được sử dụng để xây dựng các máy bộ nhớ cục bộ (LMM): xử lý các đơn vị cùng với ký ức cục bộ của chúng được kết nối với lá của cây béo, vì vậy rằng một thông điệp từ đơn vị xử lý này đến đơn vị xử lý khác trước tiên sẽ di chuyển từ cây đến

tổ tiên chung nhỏ nhất của hai đơn vị xử lý và sau đó xuống cây đơn vị xử lý đích.

2.5 Độ phức tạp tính toán song song

Để kiểm tra sự phức tạp của các vấn đề tính toán và sự song song của chúng thuật toán, chúng ta cần một số khái niệm cơ bản mới. Bây giờ chúng tôi sẽ giới thiệu một vài trong số này.

2.5.1 Các trường hợp vấn đề và kích thước của chúng

Cho Π là một bài toán tính toán. Trong thực tế chúng ta thường phải đối mặt với một trường hợp cụ thể của vấn đề Π . Thể hiện được lấy từ Π bằng cách thay thế các biến trong định nghĩa của Π với dữ liệu thực tế. Vì điều này có thể được thực hiện ở nhiều nơi theo nhiều cách, mỗi cách dẫn đến một trường hợp Π khác nhau, ta thấy rằng bài toán Π có thể được xem như một tập hợp tất cả các trường hợp cụ thể có của Π . Với mỗi trường hợp π của Π chúng ta có thể liên kết một số tự nhiên mà chúng ta gọi là kích thước của thể hiện π và ký hiệu là

kích thước(π).

Một cách không chính thức, $\text{size}(\pi)$ gần như là lượng không gian cần thiết để biểu diễn π trong một số cách mà máy tính có thể truy cập được và trong thực tế, nó phụ thuộc vào bài toán Π . Ví dụ: nếu chúng ta chọn $\Pi \equiv$ "sắp xếp một dãy số hữu hạn cho trước," thì $\pi \equiv$ "sắp xếp 0 9 2 7 4 5 6 3" là một thể hiện của Π và $\text{size}(\pi) = 8$, số lượng số cần sắp xếp. Tuy nhiên, nếu $\Pi \equiv$ "N có phải là số nguyên tố không?" thì "17 có phải là số nguyên tố không?"

số?" là một thể hiện π của Π với $\text{size}(\pi) = 5$, số bit trong hệ nhị phân biểu diễn của 17. Và nếu Π là bài toán về đồ thị thì kích thước của một thể hiện của Π thường được định nghĩa là số nút trong biểu đồ thực tế.

Tại sao chúng ta cần kích thước của phiên bản? Khi chúng ta kiểm tra tốc độ của thuật toán A đối với bài toán Π , chúng ta thường muốn biết thời gian thực hiện của A phụ thuộc vào kích thước của các thể hiện của Π được nhập vào A. Chính xác hơn, chúng ta muốn tìm một hàm

$T(n)$

có giá trị tại n sẽ biểu thị thời gian thực hiện của A trên các trường hợp có kích thước n . Như một thực tế là chúng ta quan tâm chủ yếu đến tốc độ tăng trưởng của $T(n)$, nghĩa là làm thế nào $T(n)$ tăng nhanh khi n tăng.

Ví dụ: nếu chúng ta tìm thấy $T(n) = n$ thì thời gian thực hiện của A là hàm tuyến tính của n , vì vậy nếu chúng ta tăng gấp đôi kích thước của các trường hợp có vấn đề thì thời gian thực hiện của A cũng tăng gấp đôi.

Tổng quát hơn, nếu chúng ta tìm thấy $T(n) = n \cdot \text{const}(\text{const} \neq 1)$, thì thời gian thực hiện của A là hàm đa thức của n ; nếu bây giờ chúng ta tăng gấp đôi kích thước của các trường hợp có vấn đề thì Thời gian thực hiện của A nhân với 2^{const} . Tuy nhiên, nếu chúng ta tìm thấy $T(n) = 2^n$, là hàm số mũ của n , khi đó mọi thứ trở nên kịch tính: tăng gấp đôi kích thước n của trường hợp có vấn đề khiến A chạy lâu hơn gấp 2 lần! Vì vậy, tăng gấp đôi kích thước từ 10 lên 20 rồi lên 40 và 80, thời gian thực hiện của A tăng 210 ($\approx$ ngàn) lần, sau đó là 220 ($\approx$ triệu) lần, và cuối cùng là 240 ($\approx$ ngàn tỷ) lần.

2.5.2 Số lượng đơn vị xử lý so với kích thước của các trường hợp sự cố

Trong Phần 2.1, chúng tôi đã xác định thời gian thực hiện song song T_{par} , tăng tốc S và hiệu quả E của chương trình song song P để giải bài toán Π trên máy tính $C(p)$ với p các đơn vị xử lý. Chúng ta hãy tăng cường các định nghĩa này để chúng liên quan đến kích thước n trong số các trường hợp của Π . Như trước đây, chương trình P để giải Π và máy tính $C(p)$ được hiểu ngầm nên chúng ta bỏ qua các chỉ số tương ứng để đơn giản hóa ký hiệu. Chúng ta thu được thời gian thực hiện song song $T_{\text{par}}(n)$, tăng tốc $S(n)$, và hiệu quả $E(n)$ của việc giải các trường hợp của Π có kích thước n :

$$\begin{aligned} S(n) \\ \text{def} = T_{\text{seq}}(n) / T_{\text{par}}(n), \end{aligned}$$

$$\begin{aligned} E(n) \\ \text{def} = S(n) \cdot P. \end{aligned}$$

Vì vậy, chúng ta hãy chọn một n tùy ý và giả sử rằng chúng ta chỉ quan tâm đến việc giải trường hợp của Π có kích thước là n . Bây giờ, nếu có quá ít đơn vị xử lý trong $C(p)$, tức là p quá nhỏ, khả năng song song tiềm tàng trong chương trình P sẽ không được khai thác hết trong quá trình thực thi P trên $C(p)$, và điều này sẽ phản ánh tốc độ thấp của $S(n)$ của P . Tương tự, nếu $C(p)$ có quá nhiều đơn vị xử lý, tức là p quá lớn, một số các đơn vị xử lý sẽ không hoạt động trong quá trình thực hiện chương trình P , và một lần nữa điều này sẽ phản ánh tốc độ tăng trưởng của P . Điều này đặt ra câu hỏi sau đây rõ ràng là đáng được xem xét thêm:

Có bao nhiêu đơn vị xử lý p nên có $C(p)$, sao cho, với tất cả các trường hợp Π có kích thước n , tốc độ tăng tốc của P sẽ là tối đa?

Thật hợp lý khi cho rằng câu trả lời sẽ phụ thuộc phần nào vào loại C , nghĩa là, trên mô hình đa bộ xử lý (xem Phần 2.3) bên dưới máy tính song song C . Cho đến khi chúng ta chọn mô hình đa bộ xử lý, chúng ta có thể không nhận được câu trả lời có giá trị thực tiễn cho câu hỏi trên. Tuy nhiên, chúng ta có thể đưa ra một số khái niệm chung các quan sát đúng cho bất kỳ loại C nào. Đầu tiên hãy quan sát rằng, nói chung, nếu chúng ta cho n lớn lên thì p cũng phải lớn lên; nếu không thì p cuối cùng sẽ trở thành tương đối quá nhỏ đến n , do đó làm cho $C(p)$ không có khả năng khai thác triệt để tính song song tiềm năng của P . Do đó, chúng ta có thể xem p , số lượng đơn vị xử lý cần thiết để tăng tốc tối đa, là một hàm nào đó của n , kích thước của trường hợp vấn đề tại tay. Ngoài ra, trực giác và thực tiễn cho chúng ta biết rằng một trường hợp lớn hơn của một vấn đề yêu cầu ít nhất nhiều đơn vị xử lý như yêu cầu của một đơn vị nhỏ hơn. Tóm lại, chúng tôi có thể thiết lập

$$p = f(n),$$

trong đó $f: \mathbb{N} \rightarrow \mathbb{N}$ là một hàm không giảm nào đó, tức là $f(n) \leq f(n+1)$, với mọi n .

Thứ hai, chúng ta hãy xem xét $f(n)$ có thể tăng trưởng nhanh như thế nào khi n tăng trưởng? Giả sử rằng $f(n)$ tăng theo cấp số nhân. Vâng, các nhà nghiên cứu đã chứng minh rằng nếu có số mũ. Về cơ bản có nhiều đơn vị xử lý trong một máy tính song song thì điều này nhất thiết phải kéo dài đường dẫn liên lạc giữa một số trong số họ. Vì một số quá trình giao tiếp các đơn vị trở nên xa nhau theo cấp số nhân, thời gian liên lạc giữa chúng tăng lên tương ứng và cuối cùng làm hỏng những vấn đề về mặt lý thuyết tăng tốc có thể đạt được. Lý do cho tất cả những điều đó về cơ bản là ở không gian 3 chiều thực sự của chúng ta, không gian, bởi vì

- mỗi đơn vị xử lý và mỗi liên kết truyền thông chiếm một số dung lượng khác 0 ume không gian, và
- đường kính của quả cầu nhỏ nhất chứa nhiều phép xử lý theo cấp số nhân các đơn vị và liên kết truyền thông cũng theo cấp số nhân.

Tóm lại, số lượng đơn vị xử lý theo cấp số nhân là không thực tế và dẫn đến lý thuyết

những tình huống khó xử.

Giả sử bây giờ $f(n)$ tăng trưởng đa thức, tức là f là hàm đa thức của n . Giải tích cho chúng ta biết rằng nếu $\text{poly}(n)$ và $\exp(n)$ là đa thức và là hàm mũ tương ứng, khi đó có $n' > 0$ sao cho $\text{poly}(n) < \exp(n)$ với mọi $n > n'$; nghĩa là, $\text{poly}(n)$ cuối cùng bị chi phối bởi $\exp(n)$. Nói cách khác, chúng tôi nói rằng hàm đa thức $\text{poly}(n)$ tiệm cận tăng chậm hơn hàm mũ $\exp(n)$. Lưu ý rằng $\text{poly}(n)$ và $\exp(n)$ là hai hàm tùy ý của n . Vì vậy, chúng ta có $f(n) = \text{poly}(n)$ và do đó số lượng đơn vị xử lý là

$$p = \text{poly}(n),$$

trong đó $\text{poly}(n)$ là hàm đa thức của n . Ở đây chúng ta ngầm loại bỏ đa thức chức năng của mức độ lớn "không hợp lý", ví dụ: $n!00$. Thực sự, chúng tôi đang hy vọng mức độ thấp hơn nhiều, chẳng hạn như 2, 3, 4 hoặc hơn, sẽ mang lại kết quả thực tế và giá cả phải chăng

số p của đơn vị xử lý.

Tóm lại, chúng ta đã có được câu trả lời cho câu hỏi trên – bởi vì tính tổng quát của C và Π , và do những hạn chế do tự nhiên và kinh tế áp đặt – không như mong đợi của chúng tôi. Tuy nhiên, câu trả lời cho chúng ta biết rằng p phải là một số

hàm đa thức (ở mức độ vừa phải) của n .

Chúng ta sẽ áp dụng điều này cho Định lý 2.1 (tr.16) ngay trong phần tiếp theo.

2.5.3 Lớp NC của các bài toán có thể song song hóa hiệu quả

Giả sử P là thuật toán giải bài toán Π trên $\text{CRCW-PRAM}(p)$. Theo Định lý 2.1, việc thực thi P trên $\text{EREW-PRAM}(p)$ sẽ tối đa là $O(\log p)$ -lần chậm hơn trên $\text{CRCW-PRAM}(p)$. Chúng ta hãy sử dụng những quan sát từ phần trước và yêu cầu $p = \text{poly}(n)$. Suy ra $\log p = \log \text{poly}(n) = O(\log n)$. Đến đánh giá cao lý do tại sao, xem Bài tập ở Phần 2.7.

Kết hợp với Định lý 2.1 điều này có nghĩa là với $p = \text{poly}(n)$ việc thực hiện P trên EREW-PRAM(p) sẽ chậm hơn nhiều nhất là $O(\log n)$ lần so với trên CRCW-PRAM(p). Nhưng điều này cũng cho chúng ta biết rằng, khi $p = \text{poly}(n)$, việc chọn một mô hình từ các mô hình

CRCW-PRAM(p), CREW-PRAM(p) và EREW-PRAM(p) để thực thi chương trình ảnh hưởng đến thời gian thực hiện của chương trình theo hệ số cấp $O(\log n)$, trong đó n là kích thước của các trường hợp vấn đề cần được giải quyết. Nói cách khác:

Thời gian thực hiện của một chương trình không thay đổi quá nhiều khi chúng tôi chọn biến thể PRAM sẽ thực thi nó.

Điều này thúc đẩy chúng tôi giới thiệu một lớp các bài toán tính toán chứa tất cả các vấn đề có thuật toán song song “nhanh” đòi hỏi số lượng “hợp lý” các đơn vị xử lý. Nhưng “nhanh” và “hợp lý” thực sự có nghĩa là gì? Chúng tôi đã thấy ở phần trước rằng số lượng đơn vị xử lý là hợp lý nếu nó là đa thức ở n . Về ý nghĩa của “nhanh”, một thuật toán song song được coi là nhanh nếu thời gian thực hiện song song của nó là đa logarit tính bằng n . Điều đó thì tốt, nhưng làm sao bây giờ “đa logarit” nghĩa là gì? Đây là định nghĩa.

Định nghĩa 2.1 Một hàm là đa thức trong n nếu nó là đa thức trong $\log n$, tức là, nếu nó là $a_k(\log n)^k + a_{k-1}(\log n)^{k-1} + \dots + a_1(\log n) + a_0$, với một số $k \geq 1$.

Chúng ta thường viết $\log_i n$ thay vì $(\log n)^i$ để tránh gộp các dấu ngoặc đơn. Tổng $a_k \log_i n + a_{k-1} \log_{i-1} n + \dots + a_0$ được giới hạn tiệm cận ở trên bởi $O(\log_i n)$. Để biết lý do tại sao, hãy xem xét các bài tập ở Mục 2.7.

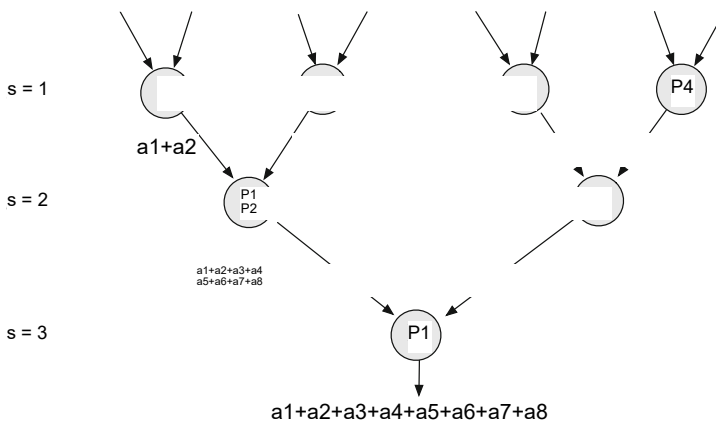
Chúng tôi sẵn sàng giới thiệu chính thức lớp vấn đề mà chúng tôi quan tâm.

Định nghĩa 2.2
Giả sử NC là lớp các bài toán tính toán có thể giải được trong thời gian đa logarit trên PRAM với số đa thức của đơn vị xử lý.

Nếu bài toán Π thuộc lớp NC thì nó có thể giải được trong thời gian song song đa logarit với nhiều đơn vị xử lý đa thức bất kể biến thể PRAM được sử dụng để giải quyết Π . Nói cách khác, lớp NC mạnh mẽ, không nhạy cảm với các biến thể của PRAM. Làm thế nào chúng ta có thể thấy điều đó? Nếu chúng ta thay thế một biến thể của PRAM bằng một biến thể khác thì theo Định lý 2.1 Π thời gian thực hiện song song $O(\log_i n)$ chỉ có thể tăng một hệ số $O(\log n)$ đến $O(\log_{i+1} n)$ vẫn là đa logarit. Tóm lại, NC là lớp các bài toán tính toán có thể song song hóa một cách hiệu quả.

Ví dụ 2.1 Giả sử chúng ta có bài toán $\Pi \equiv$ “cộng n số đã cho”.

Khi đó $\Pi \equiv$ “cộng các số 10, 20, 30, 40, 50, 60, 70, 80” là một thể hiện của $\text{size}(\Pi) = 8$



Hình.2.16 Cộng tám số song song với bốn bộ xử lý

của vấn đề Π . Bây giờ chúng ta tập trung vào tất cả các trường hợp có kích thước 8, nghĩa là các trường hợp của

dạng $\pi =$ "cộng các số $a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$."

Thuật toán tuần tự nhanh nhất để tính toán $\text{thesuma}_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 + a_8$ yêu cầu $T_{\text{seq}}(8) = 7$ bước, mỗi bước cộng số tiếp theo vào tổng của những cái trước đó.

Tuy nhiên, song song đó các số $a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$ có thể được tổng hợp bằng chỉ $T_{\text{par}}(8) = 3$ bước song song sử dụng 8

$2 = 4$ đơn vị xử lý giao tiếp trong một

mô hình cây như mô tả trong hình.2.16. Ở bước đầu tiên, $s = 1$, mỗi đơn vị xử lý thêm hai số đầu vào liên kế. Trong mỗi bước tiếp theo, $s \leq 2$, hai bước liên kế trước đó kết quả từng phần được thêm vào để tạo ra một kết quả từng phần mới, kết hợp. Sự kết hợp này

kết quả từng phần theo kiểu cây tiếp tục cho đến $2s+1 > 8$. Ở bước đầu tiên, $s = 1$, tất cả bốn đơn vị xử lý đều tham gia tính toán; ở bước $s = 2$, hai bộ xử lý (P_3 và P_4) bắt đầu chạy không tải; và ở bước $s = 3$, ba đơn vị xử lý (P_2, P_3 và P_4) không hoạt động.

Nói chung, các trường hợp $\pi(n)$ của Π có thể được giải trong thời gian song song $T_{\text{par}} = \lceil \log n \rceil =$

$O(\log n)$ với $\lceil \log n \rceil$

$2 \lceil \log n \rceil = O(n)$ các đơn vị xử lý giao tiếp theo kiểu cây tương tự nhận biến. Do đó, $\Pi \in NC$ và tốc độ tăng tốc liên quan là $S(n) = T_{\text{seq}}(n) / T_{\text{par}}(n) = O(n / \log n) = \Omega(n / \log n)$.

Lưu ý rằng, trong ví dụ trên, hiệu quả của phép cộng song song giống như cây của n số khá thấp, $E(n) = O(\log n)$.

Lý do cho điều này là hiển nhiên: chỉ một nửa số đơn vị xử lý tham gia vào bước song song sẽ được tham gia vào bước tiếp theo song song bước $s + 1$, trong khi tất cả các đơn vị xử lý khác sẽ không hoạt động cho đến hết tính toán. Vấn đề này sẽ được giải quyết trong phần tiếp theo của Định lý Brent.

2.6 Các định luật và định lý tính toán song song

Trong phần này chúng tôi mô tả định lý Brent, định lý này rất hữu ích trong việc ước tính giới hạn dưới về số lượng đơn vị xử lý cần thiết để duy trì một số lượng nhất định độ phức tạp thời gian song song. Sau đó chúng ta tập trung vào định luật Amdahl, được sử dụng để dự đoán tốc độ tăng tốc về mặt lý thuyết của một chương trình song song có các phần khác nhau cho phép tăng tốc khác nhau.

2.6.1 Định lý Brent

Định lý Brent cho phép chúng ta định lượng hiệu suất của một chương trình song song khi số lượng đơn vị xử lý giảm. Cho M là một PRAM có kiểu tùy ý và chứa số lượng không xác định các đơn vị xử lý. Cụ thể hơn, chúng tôi giả định rằng số lượng đơn vị xử lý luôn luôn đủ để đáp ứng tất cả các nhu cầu của bất kỳ chương trình song song nào. Khi một chương trình song song P chạy trên M, số thao tác khác nhau của P là được thực hiện ở mỗi bước bởi các đơn vị xử lý khác nhau của M. Giả sử rằng tổng cộng

W

các hoạt động được thực hiện trong quá trình thực hiện song song P trên M (W còn được gọi là công việc của P) và biểu thị thời gian chạy song song của P trên M bằng

$T_{par}, M(P).$

Bây giờ chúng ta hãy giảm số lượng đơn vị xử lý của M xuống một số cố định

P

và biểu thị máy thu được có số lượng đơn vị xử lý giảm đi bằng

R.

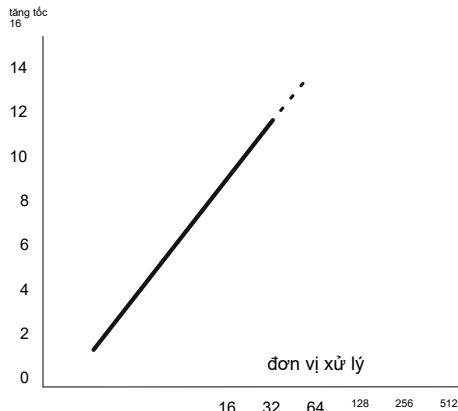
R là PRAM cùng loại với M, có thể sử dụng trong mỗi bước hoạt động của nó với tốc độ hầu hết các đơn vị xử lý p. Bây giờ chúng ta chạy P trên R. Nếu đơn vị xử lý p không thể hỗ trợ, trong mỗi bước của thực thi, tất cả khả năng song song tiềm năng của P, sau đó là thời gian chạy song song của P trên R,

$T_{par}, R(P),$

có thể lớn hơn $T_{par}, M(P)$. Bây giờ câu hỏi đặt ra là: Chúng ta có thể định lượng $T_{par}, R(P)$ không? Câu trả lời được đưa ra bởi Định lý Brent trong đó phát biểu rằng

$$T_{par}, R(P) = O \frac{W}{p} + T_{par}, M(P)$$

Hình.2.17 Dự kiến (tuyến tính) tăng tốc như một chức năng của số lượng đơn vị xử lý



Chứng minh Gọi W_i là số thao tác của P được M thực hiện ở bước thứ i và $T := T_{par}, M(P)$. Sau đó T
 $i=1$ $W_i = W$. Để thực hiện các thao tác W_i ở bước thứ i
của nhu cầu M, R $\square$
 W_i các bước. Vậy số bước mà R thực hiện trong thời gian $\square$
thực hiện P là $T_{par}, R(P) = T$ $p + 1$
 $i=1$ $\square$
 W_i $\square 1$
 $i=1$ $W_i + T =$
 W
 $p + T_{par}, M(P)$.
 $\square$
 $\square$

Ứng dụng của Định lý Brent

Định lý Brent rất hữu ích khi chúng ta muốn giảm số lượng đơn vị xử lý như càng nhiều càng tốt trong khi vẫn giữ được độ phức tạp về thời gian song song. Ví dụ, chúng tôi có thấy trong Ví dụ 2.1 trên trang 34 rằng chúng ta có thể tính tổng n số trong thời gian song song $O(\log n)$ với các đơn vị xử lý $O(n)$. Liệu chúng ta có thể làm điều tương tự với việc xử lý ít tiem cận hơn không? đơn vị? Vâng, chúng tôi có thể. Định lý Brent cho chúng ta biết rằng các đơn vị xử lý $O(n/\log n)$ là đủ để tính tổng n số trong thời gian song song $O(\log n)$. Xem bài tập ở phần 2.7.

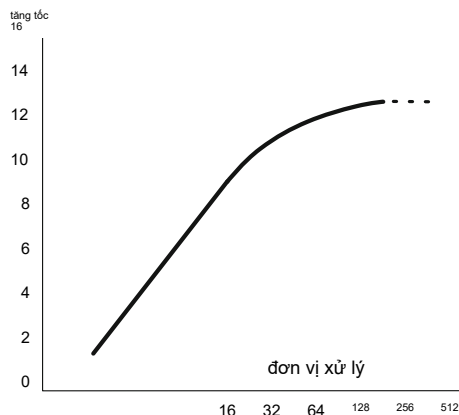
2.6.2 Định luật Amdahl

Theo trực giác, chúng ta kỳ vọng rằng việc tăng gấp đôi số lượng đơn vị xử lý sẽ giảm một nửa thời gian thực hiện song song; và tăng gấp đôi số lượng đơn vị xử lý một lần nữa nên giảm một nửa thời gian thực hiện song song một lần nữa. Nội cách khác, chúng tôi mong đợi rằng việc tăng tốc độ song song hóa là một hàm tuyến tính của số lượng xử lý đơn vị (xem hình.2.17).

Tuy nhiên, tăng tốc tuyến tính từ việc song song hóa chỉ là một điều tối ưu mong muốn khó có thể trở thành hiện thực lắm. Quả thực, trong thực tế rất ít thuật toán song song đạt được nó. Hầu hết các chương trình song song đều có tốc độ tăng tốc gần như tuyến tính đối với các chương trình nhỏ.

số phần tử xử lý, sau đó làm phẳng thành một giá trị không đổi cho số lượng lớn số phần tử xử lý (xem Hình 2.18).

Hình.2.18 Tăng tốc thực tế như một hàm của số đơn vị xử lý



Thiết lập sân khấu

Làm thế nào chúng ta có thể giải thích hành vi bất ngờ này? Mạnh mỗi cho câu trả lời sẽ là thu được từ hai ví dụ đơn giản.

• Ví dụ 1. Cho P là một chương trình tuần tự xử lý các tập tin từ đĩa như sau:

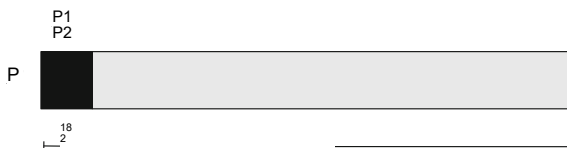
- P là dãy gồm hai phần, $P = P1P2$;
- P1 quét thư mục của đĩa, tạo danh sách tên tập tin và trao liệt kê sang P2;
- P2 chuyển từng tệp từ danh sách đến bộ xử lý để xử lý tiếp.

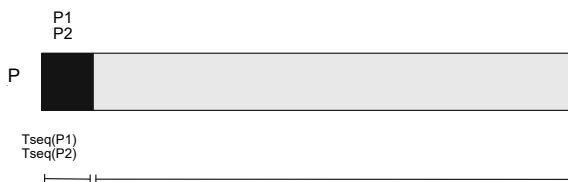
Lưu ý: Không thể tăng tốc P1 bằng cách thêm các đơn vị xử lý mới, vì việc quét thư mục đĩa thực chất là quá trình tuần tự. Ngược lại, P2 có thể được tăng tốc bằng cách thêm các đơn vị xử lý mới; ví dụ: mỗi tệp có thể được chuyển đến một tệp riêng biệt đơn vị xử lý. Tóm lại, một chương trình tuần tự có thể được xem như một chuỗi gồm hai các phần khác nhau về khả năng song song hóa của chúng, tức là khả năng tuân theo sự song song hóa.

• Ví dụ 2. Cho P như trên. Giả sử rằng việc thực hiện (tuần tự) của P mất 20 phút, trong đó các giá trị sau được giữ (xem Hình 2.19):

- P1 không song song chạy trong 2 phút;
- P2 song song chạy 18 phút.

Hình.2.19 P bao gồm một P1 không song song và có thể song song hóa P2. Trên một bộ xử lý, P1 chạy 2 phút và P2 chạy 18 phút





Hình.2.20 P bao gồm P1 không song song và P2 song song. Trên một đơn vị xử lý P1 yêu cầu thời gian $T_{seq}(P1)$ và P2 yêu cầu thời gian $T_{seq}(P2)$ để hoàn thành

Lưu ý: vì chỉ P2 mới có thể hưởng lợi từ các đơn vị xử lý bổ sung nên việc thực thi song song thời gian hoạt động $T_{seq}(P)$ của toàn bộ P không thể nhỏ hơn thời gian $T_{seq}(P1)$ được thực hiện bởi phần P1 không thể song song (nghĩa là 2 phút), bất kể số lượng phần bổ sung các đơn vị kết thúc tham gia vào việc thực hiện song song P. Tóm lại, nếu các phần của chuỗi tuần tự chương trình khác nhau về khả năng song song tiềm năng, chúng khác nhau về khả năng tăng tốc do số lượng đơn vị xử lý tăng lên, do đó tốc độ của toàn bộ chương trình sẽ phụ thuộc vào thời gian chạy tuần tự của chúng.

Những manh mối mà các ví dụ trên đưa ra được tóm tắt lại như sau: Trong Nói chung, một chương trình P được thực thi bởi một máy tính song song có thể được chia thành hai phần,

- phần P1 không được hưởng lợi từ nhiều đơn vị xử lý và
- phần P2 được hưởng lợi từ nhiều đơn vị xử lý;
- bên cạnh lợi ích của P2, còn có thời gian thực hiện tuần tự của ảnh hưởng P1 và P2 thời gian thực hiện song song của toàn bộ P (và do đó tăng tốc P).

Đạo hàm

Bây giờ chúng ta sẽ đánh giá một cách định lượng mức độ tăng tốc của P phụ thuộc vào P1 và P2 như thế nào thời gian thực hiện tuần tự và khả năng song song hóa và khai thác của chúng của nhiều đơn vị xử lý.

Gọi $T_{seq}(P)$ là thời gian thực hiện tuần tự của P. Vì $P = P1P2$ nên một chuỗi của phần P1 và P2, ta có

$$T_{seq}(P) = T_{seq}(P1) + T_{seq}(P2),$$

trong đó $T_{seq}(P1)$ và $T_{seq}(P2)$ là thời gian thực hiện tuần tự của P1 và P2, cụ thể: một cách tích cực (xem Hình 2.20).

Khi chúng tôi thực sự sử dụng các đơn vị xử lý bổ sung để thực hiện song song P, đó là việc thực thi P2 được tăng tốc bởi một số hệ số $s > 1$, trong khi việc thực thi của P1 không được hưởng lợi từ các đơn vị xử lý bổ sung. Nói cách khác, việc thực hiện thời gian của P2 giảm từ $T_{seq}(P2)$ xuống 1

s $T_{seq}(P2)$, trong khi thời gian thực hiện của P1 vẫn giữ nguyên, $T_{seq}(P1)$. Vì vậy, sau khi sử dụng thêm các đơn vị xử lý thời gian thực hiện song song $T_{par}(P)$ của toàn bộ chương trình P là

$$T_{par}(P) = T_{seq}(P1) + \frac{1}{s} T_{seq}(P2).$$

Tốc độ tăng tốc $S(P)$ của toàn bộ chương trình P bây giờ có thể được tính từ định nghĩa,

$$S(P) = \frac{T_{\text{seq}}(P)}{T_{\text{par}}(P)}.$$

Chúng ta có thể dừng ở đây; tuy nhiên, người ta thường biểu thị $S(P)$ theo b , phân số của $T_{\text{seq}}(P)$ trong đó việc song song hóa P là có lợi. Trong trường hợp của chúng tôi

$$b = \frac{T_{\text{seq}}(P_2)}{T_{\text{seq}}(P)}.$$

Thay biểu thức này vào biểu thức của $S(P)$, cuối cùng chúng ta thu được Định luật Amdahl

$$S(P) = \frac{1}{1 - b + \frac{b}{s}}.$$

Một số nhận xét về Định luật Amdahl

Nói một cách chính xác, tốc độ tăng tốc trong Định luật Amdahl là một hàm của ba biến,

P , b và s nên sẽ được ký hiệu phù hợp hơn là $S(P, b, s)$. Đây b là phần thời gian trong đó việc thực hiện tuần tự của P có thể được hưởng lợi từ nhiều đơn vị xử lý. Nếu có nhiều đơn vị xử lý thực sự có sẵn và bị P khai thác thì phần P khai thác chúng được tăng tốc bởi hệ số $s > 1$. Vì s chỉ là tăng tốc một phần chương trình P , tăng tốc toàn bộ P không thể lớn hơn s ; cụ thể là nó được cho bởi $S(P)$ của Định luật Amdahl.

Từ Định luật Amdahl chúng ta thấy rằng

$$S < \frac{1}{1 - b},$$

điều này cho chúng ta biết rằng một phần nhỏ của chương trình không thể song song hóa sẽ hạn chế tăng tốc tổng thể có sẵn từ song song hóa. Ví dụ: tăng tốc tổng thể S mà chương trình P trong Hình.2.19 có thể đạt được bằng cách song song hóa phần P_2 được giới hạn ở trên bởi $S < \frac{1}{1 - 18}$

$$20 = 10.$$

Lưu ý rằng trong cách suy ra Định luật Amdahl không nói gì về kích thước của trường hợp vấn đề được giải quyết bằng chương trình P . Người ta ngầm giả định rằng vấn đề

instance vẫn giữ nguyên và điều duy nhất chúng tôi thực hiện là song song hóa P và sau đó áp dụng P song song trên cùng một trường hợp bài toán. Như vậy, Định luật Amdahl chỉ áp dụng cho trường hợp kích thước của trường hợp vấn đề là cố định.

Định luật Amdahl tại nơi làm việc

Giả sử rằng 70% quá trình thực thi chương trình có thể được tăng tốc nếu chương trình được song song hóa

và chạy trên 16 đơn vị xử lý thay vì một đơn vị. Tăng tốc tối đa là bao nhiêu

toàn bộ chương trình có thể đạt được? Tăng tốc tối đa là bao nhiêu nếu chúng ta tăng số lượng đơn vị xử lý lên 32, sau đó lên 64, rồi lên 128?

Trong trường hợp này chúng ta có $b = 0,7$, phần thực hiện tuần tự có thể được song song; và $\frac{1}{1 - b} = 0,3$, tỷ lệ tính toán không thể song song hóa.

Tốc độ tăng tốc của phân số song song là s. Tất nhiên, $s \leq p$, trong đó p là số lượng đơn vị xử lý Theo định luật Amdahl tốc độ của toàn bộ chương trình là

$$S = \frac{s}{1 - b + \frac{s}{b}} = \frac{s}{0,3 + \frac{s}{0,7}} \quad 0,3 + \frac{-16}{0,7} = 2,91.$$

Nếu chúng ta nhân đôi số lượng đơn vị xử lý lên 32 thì chúng ta sẽ thấy rằng số lượng tối đa tăng tốc có thể đạt được là 3,11:

$$S = \frac{s}{1 - b + \frac{s}{b}} = \frac{s}{0,3 + \frac{s}{0,7}} \quad 0,3 + \frac{-32}{0,7} = 3,11,$$

và nếu chúng ta tăng gấp đôi nó một lần nữa lên 64 đơn vị xử lý, thì mức tối đa có thể đạt được tăng tốc trở thành 3,22:

$$S = \frac{s}{1 - b + \frac{s}{b}} = \frac{s}{0,3 + \frac{s}{0,7}} \quad 0,3 + \frac{-64}{0,7} = 3,22.$$

Cuối cùng, nếu chúng ta tăng gấp đôi số lượng đơn vị xử lý lên 128 thì mức tối đa tăng tốc chúng ta có thể đạt được là

$$S = \frac{s}{1 - b + \frac{s}{b}} = \frac{s}{0,3 + \frac{s}{0,7}} \quad 0,3 + \frac{-128}{0,7} = 3,27.$$

Trong trường hợp này, việc tăng gấp đôi sức mạnh xử lý chỉ cải thiện được tốc độ một chút.

Vì vậy, sử dụng nhiều đơn vị xử lý hơn không hẳn là phương pháp tối ưu.

Lưu ý rằng điều này tuân thủ việc tăng tốc thực tế của các chương trình thực tế như chúng tôi có được mô tả trong hình.2.18.

□ Tổng quát hóa định luật Amdahl

Cho đến bây giờ chúng ta giả định rằng chỉ có hai phần của một chương trình nhất định, trong đó

một cái không thể hưởng lợi từ nhiều đơn vị xử lý và cái kia thì có thể. Bây giờ chúng ta giả sử

rằng chương trình là một chuỗi gồm ba phần, mỗi phần có thể được hưởng lợi từ nhiều đơn vị xử lý. Mục tiêu của chúng tôi là tăng tốc toàn bộ chương trình khi chương trình được thực hiện bởi nhiều đơn vị xử lý.

Vì vậy, giả sử $P = P1P2P3$ là một chương trình gồm ba phần P1, P2 và P3. Xem hình 2.21. Gọi Tseq(P1) là thời gian trong đó việc thực hiện tuần tự của

Hình.2.21 P bao gồm ba cách khác nhau phân song song P



P thực hiện phần P1. Tương tự, chúng ta định nghĩa Tseq(P2) và Tseq(P3). Sau đó thời gian thực hiện tuần tự của P là

$$T_{\text{seq}}(P) = T_{\text{seq}}(P1) + T_{\text{seq}}(P2) + T_{\text{seq}}(P3).$$

Nhưng chúng tôi muốn chạy P trên một máy tính song song. Giả sử rằng phân tích của P cho thấy rằng P1 có thể được song song hóa và tăng tốc trên máy song song với hệ số $s1 > 1$. Tương tự, P2 và P3 có thể được tăng tốc theo hệ số $s2 > 1$ và $s3 > 1$ tương ứng. Vì vậy, chúng ta song song hóa P bằng cách song song hóa từng phần trong số ba phần P1, P2 và P3 và chạy P trên máy song song. Việc thực hiện song song P mất thời gian Tpar(P), trong đó $T_{\text{par}}(P) = T_{\text{par}}(P1) + T_{\text{par}}(P2) + T_{\text{par}}(P3)$. Nhưng $T_{\text{par}}(P1) = 1$

$s1 T_{\text{seq}}(P1)$, và tương tự cho $T_{\text{par}}(P2)$ và $T_{\text{par}}(P3)$. Nó theo sau đó

$$T_{\text{par}}(P) = \frac{1}{s1} T_{\text{seq}}(P1) + \frac{1}{s2} T_{\text{seq}}(P2) + \frac{1}{s3} T_{\text{seq}}(P3).$$

Bây giờ sự tăng tốc của P có thể dễ dàng được tính từ định nghĩa của nó, $S(P) = T_{\text{seq}}(P) / T_{\text{par}}(P)$. Chúng ta có thể thu được một biểu thức giàu thông tin hơn cho $S(P)$. Gọi $b1$ là phân số của $T_{\text{seq}}(P)$ trong đó việc thực thi tuần tự của P thực hiện P1, đó là, $b1 = T_{\text{seq}}(P1) / T_{\text{seq}}(P)$. Tương tự, chúng ta xác định $b2$ và $b3$. Áp dụng điều này trong định nghĩa của $S(P)$ chúng ta thu được

$$S(P) = \frac{T_{\text{seq}}(P)}{\frac{1}{s1} T_{\text{seq}}(P1) + \frac{1}{s2} T_{\text{seq}}(P2) + \frac{1}{s3} T_{\text{seq}}(P3)} = \frac{1}{\frac{b1}{s1} + \frac{b2}{s2} + \frac{b3}{s3}}$$

Khái quát hóa cho các chương trình là các chuỗi có số phần P_i tùy ý là đơn giản. Trong thực tế, các chương trình thường bao gồm một số phần có thể song song hóa và một số phần không thể song song (nối tiếp). Chúng ta dễ dàng giải quyết vấn đề này bằng cách đặt $s_i = 1$.

2.7 Bài tập

1. Cần phải tính bao nhiêu tương tác từng cặp khi giải phần n vấn đề nếu chúng ta giả định rằng các tương tác là đối xứng?
2. Đưa ra lời giải thích trực quan tại sao $T_{\text{par}} \leq p \cdot T_{\text{seq}}$, trong đó T_{par} và T_{seq} nằm ở đầu thời gian thực hiện song song và tuần tự của một chương trình, và p là số lượng đơn vị xử lý được sử dụng trong quá trình thực hiện song song.
3. Bạn có thể ước tính số lượng cấu trúc liên kết mạng khác nhau có khả năng tương tác với nhau không?
4. Giả sử bộ xử lý p P_i và mô-đun bộ nhớ M_j ? Giả sử rằng mỗi Cấu trúc liên kết phải cung cấp cho mỗi cặp (P_i, M_j) một đường dẫn giữa P_i và M_j . Theo Định lý 2.1, việc thực thi P trên EREW-PRAM(p) sẽ nhiều nhất là $O(\log p)$ -lần chậm hơn so với trên CRCW-PRAM(p). Bây giờ giả sử rằng $p = \text{poly}(n)$, trong đó n là kích thước của một thể hiện vấn đề. Chứng minh rằng $\log p = O(\log n)$.

5. Chứng minh rằng tổng $a_0 \log n + a_1 \log(n-1) + \dots + a_{n-1} \log 1$ là tiệm cận giới hạn ở trên bởi $O(\log n)$.
6. Chứng minh rằng bộ xử lý $O(n/\log n)$ đủ để tính tổng n số trong $O(\log n)$ thời gian song song. Gợi ý: Giả sử các số được tính tổng bằng một cây thuật toán song song được mô tả trong Ví dụ 2.1. Sử dụng Định lý Brent với $W = n-1$ và $T = \log n$ và quan sát điều đó bằng cách giảm số lượng đơn vị xử lý đến $p := n/\log n$, thuật toán song song dạng cây sẽ giữ lại độ song song $O(\log n)$ của nó sự phức tạp về thời gian.
7. Đúng hay sai:
- (a) Định nghĩa về thời gian thực hiện song song là: "thời gian thực hiện = tính toán thời gian hoạt động + thời gian liên lạc + thời gian nhân rồi."
- (b) Một mô hình đơn giản về thời gian truyền thông là: "thời gian truyền thông = set-thời gian hoạt động + thời gian truyền dữ liệu."
- (c) Giả sử thời gian thực hiện của một chương trình trên một bộ xử lý là T_1 , và thời gian thực hiện của cùng một chương trình song song trên bộ xử lý p là T_p . Khi đó, tốc độ tăng tốc và hiệu suất lần lượt là $S = T_1/T_p$ và $E = S/p$.
- (d) Nếu tăng tốc $S < p$ thì $E > 1$.
8. Đúng hay sai:
- (a) Nếu các đơn vị xử lý giống hệt nhau thì để giảm thiểu việc thực hiện song song thời gian, công việc (hoặc tải tính toán) của một chương trình song song phải được chia thành được chia thành các phần bằng nhau và phân bổ giữa các đơn vị xử lý.
- (b) Nếu các đơn vị xử lý khác nhau về khả năng tính toán thì để giảm thiểu mô phỏng thời gian thực hiện song song, công việc (hoặc tải tính toán) của song song chương trình phải được phân bổ đều giữa các đơn vị xử lý.
- (c) Việc tìm kiếm các phân bổ như vậy được gọi là cân bằng tải.
9. Tại sao tải của chương trình song song phải được phân bổ đều giữa các bộ xử lý?
10. Xác định bằng thông chia đôi của lưới 1D (chuỗi máy tính có biên-kết nối trực tràng), lưới 2D, lưới 3D và siêu khối.
11. Giả sử một chương trình P bao gồm một phần R có thể song song một cách lý tưởng và của phần tuần tự S ; nghĩa là $P = RS$. Trên một bộ xử lý, S chiếm 10% tổng thời gian thực hiện và trong 90% thời gian còn lại R có thể chạy trong song song.
- (a) Tốc độ tối đa có thể đạt được với số lượng bộ xử lý không giới hạn là bao nhiêu?
- (b) Luật này được gọi như thế nào?
12. Định luật Moore phát biểu rằng hiệu suất máy tính tăng gấp đôi sau mỗi 1,5 năm. Giả sử rằng hiệu suất máy tính hiện tại là $\text{Perf} = 1013$. Khi nào sẽ có, theo theo định luật này, lớn hơn 10 lần (tức là $10 \times \text{Perf}$)?
13. Bài toán Π gồm hai bài toán con Π_1 và Π_2 được giải bằng chương trình P_1 và P_2 tương ứng. Chương trình P_1 sẽ chạy 1000 giây trên máy tính C_1 và 2000 trên máy tính C_2 , trong khi P_2 sẽ yêu cầu 2000 và 3000 giây tương ứng trên C_1 và C_2 . Các máy tính được kết nối bằng khoảng cách 1000 km liên kết sợi quang dài có khả năng truyền dữ liệu với tốc độ 100 MB/giây với 10 mili giây độ trễ. Các chương trình có thể thực hiện đồng thời nhưng phải truyền (a) 10 MB dữ liệu 20.000 lần hoặc (b) 1 MB dữ liệu hai lần trong quá trình thực hiện. cái gì cấu hình tốt nhất và thời gian chạy gần đúng trong trường hợp (a) và (b) là gì?

2.8 Ghi chú thư mục

Khi trình bày các chủ đề trong Chương này, chúng tôi chủ yếu dựa vào Trobec et al. [26] và Atallah và Blanton [3]. Về mô hình tính toán tuần tự xem Robić [22]. Mạng kết nối được thảo luận rất chi tiết trong Dally và Khān [6], Duato và cộng sự. [7], Trobec [25] và Trobec et al. [26]. Sự phụ thuộc về thời gian thực hiện của các ứng dụng song song trong thế giới thực đối với hiệu suất của mạng kết nối được thảo luận trong Grama et al. [12].